Module-1

JAVA BASICS

OOP Paradigm in Java

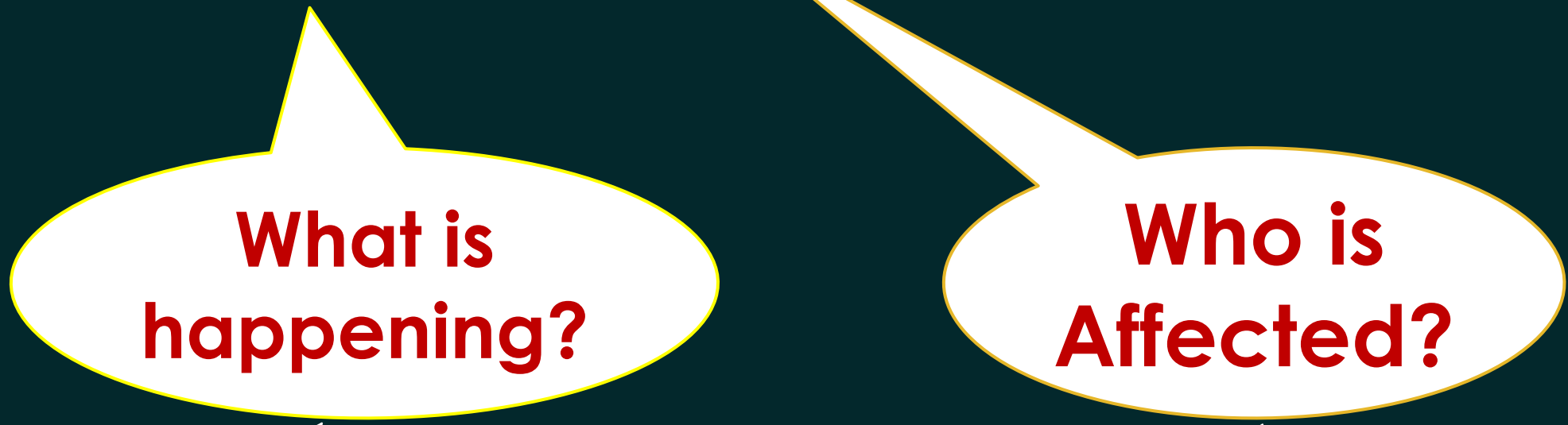
- Programming Paradigm
- Procedure Oriented Paradigm Vs OOP Paradigm
- **OOP Paradigm**
- **OOP Principles**
- Benefits of OOPs
- Introduction to Java
- History of Java

Programming Paradigm

- Programming paradigms are different methodologies or styles of programming that provide a set of principles, notions, and practices to solve computational problems.
- A programming paradigm defines the way in which a programmer structures and organizes code to build software applications.

Procedure Oriented Vs Object Oriented

- Program Code & Program Data.



Procedure Oriented Programming

Object Oriented Programming

OOP Paradigm

- OOP stands for Object-Oriented Programming.
- Object-oriented programming (OOP) is a programming paradigm that is based on the concept of objects, which are instances of classes that encapsulate data and behavior.

Classes

Objects



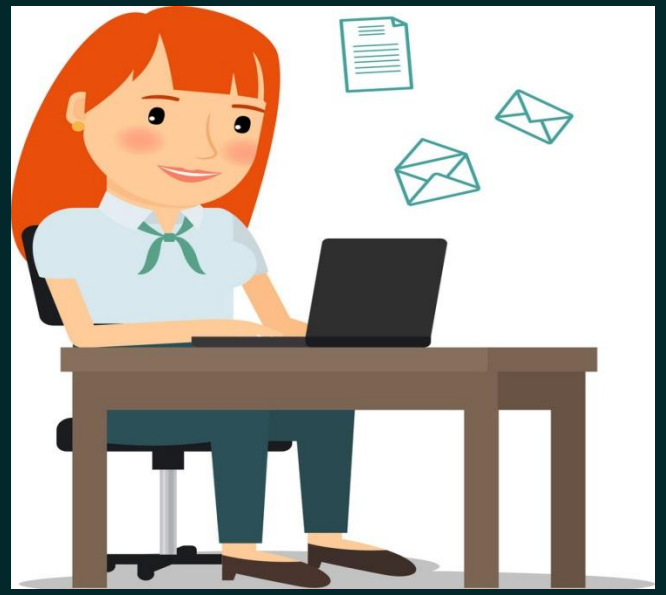
OOP Principles

- Abstraction
- Encapsulation
- Inheritance (Code Reuse)
- Polymorphism (Same method different behavior for different object)

Abstraction

- Hiding Complexities
- Examples of Abstraction
  - A man driving a car.
  - Sending an e-mail.
- Achieved through
  - Abstract class
  - Interfaces (100% Abstraction)



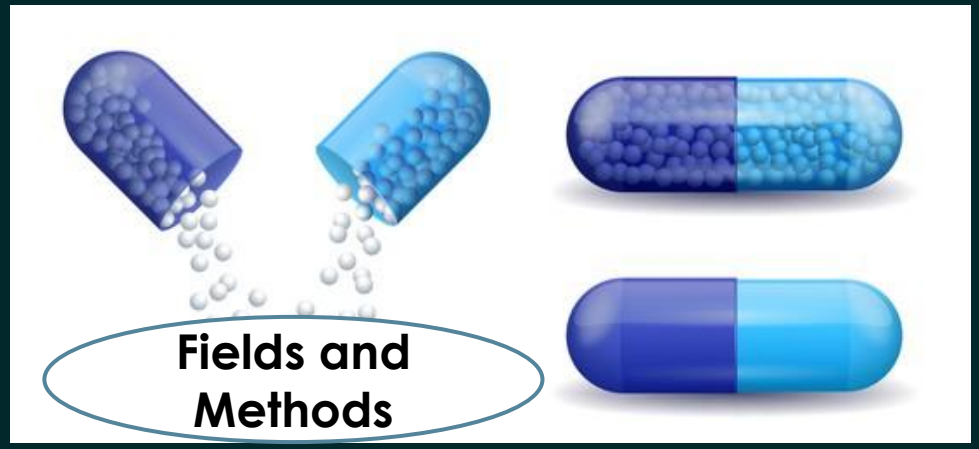


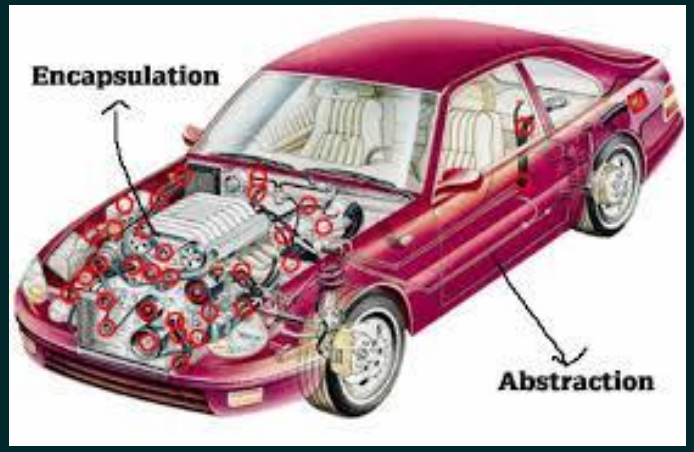
Encapsulation

- Wrapping up of fields and methods
- Data Hiding
- Example

  car
- Achieved through

  Class





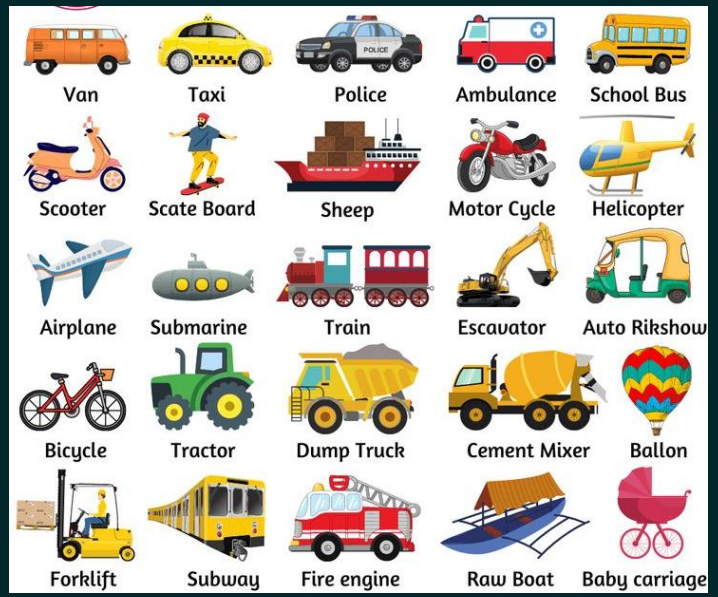
Inheritance

- Creating classes from existing classes
- Reuse of code
- Example
  - Child inheriting features of Parent
  - Vehicle class



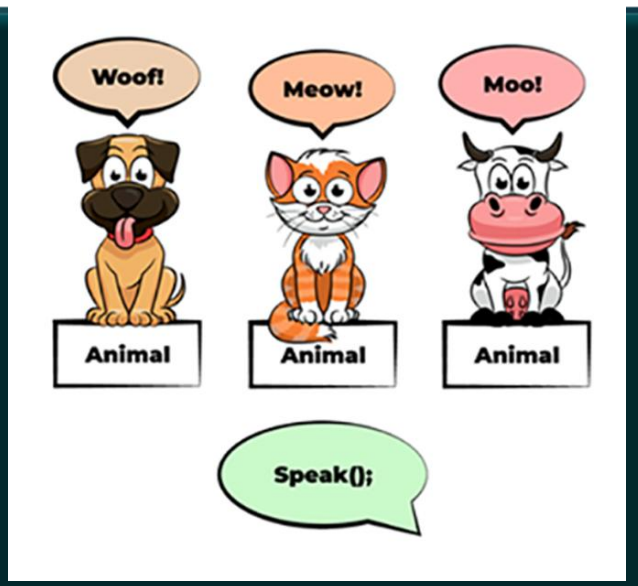


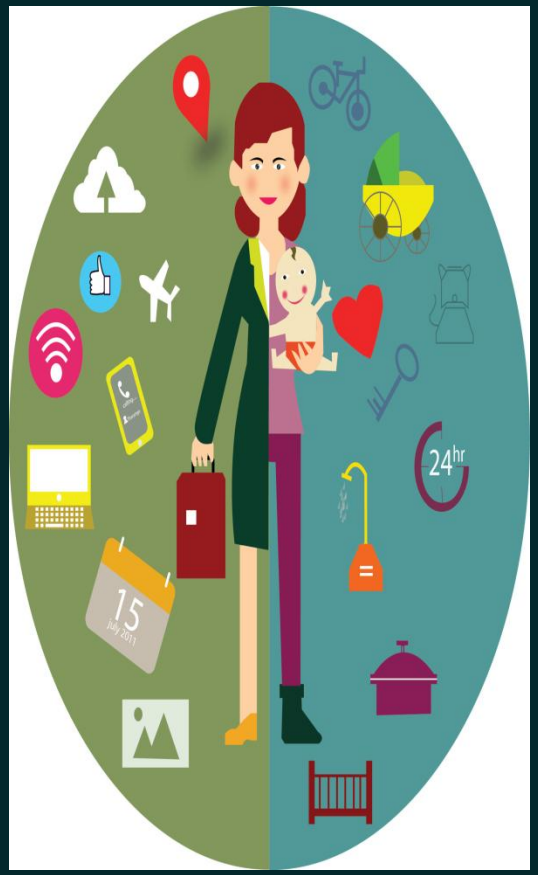




Polymorphism

- Performing a single action in different ways
- Poly-many, Morph-forms
- Example:
  - Animals talking
  - Woman playing multi roles





Benefits of OOPs

- Re-usability
- Data redundancy
- Code maintenance
- Security
- Design benefits
- Easy trouble shooting

Introduction to Java

- Java is a popular programming language, created in 1995.
- It is owned by Oracle, and more than 3 billion devices run Java.
- It is used for:
  - Mobile applications (specially Android apps)
  - Desktop applications
  - Web applications
  - Web servers and application servers
  - Games
  - Database connection

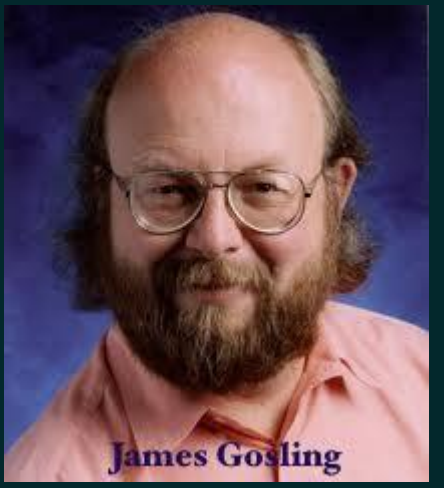
Introduction to Java

- Java works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc.)
- It is one of the most popular programming language in the world
- It is easy to learn and simple to use
- It is open-source and free
- It is secure, fast and powerful

Introduction to Java

- It has a huge community support (tens of millions of developers)
- Java is an object oriented language which gives a clear structure to programs and allows code to be reused, lowering development costs
- As Java is close to C++ and C#, it makes it easy for programmers to switch to Java

History of Java



- Java was developed by Sun Microsystems (which is now the subsidiary of Oracle) in the year 1995.
- James Gosling is known as the father of Java.
- Before Java, its name was Oak. Since Oak was already a registered company, so James Gosling and his team changed the name from Oak to Java.

Mike Sheridan, and Patrick Naughton initiated the Java language project in June 1991. The small team of sun engineers called Green Team.

History of Java

- Beginning(1991-1992) Green Project
- 1994 - Named as JAVA
- 1995 – public announcement
- Mid 1990s – Applets and WORA
- Late 1990s - Java 2 (J2SE)
- Late 1990s to Early 2000s (disputes)
- Open Sourcing (2006)

Write Once Run Any where

History of Java

- Continuous Evolution –
  - Java 5 (introducing generics)
  - Java 8 (with lambdas and the Stream API)
  - Java 11 (a long-term support release).
- Ownership by Oracle(2010)
- Java Today

Features of Java

- 12 Features

Features of Java

- Simple
- Platform Independent
- Architecture Neutral
- Portable
- Secure
- Object Oriented
- Multithreaded
- Robust
- Distributed
- Interpreted
- High Performance
- Dynamic

Simple

- Easy to learn and understand.
- Complex concepts like pointer, operator overloading of CPP are excluded.
- No steps needed to remove unreferenced objects because there is an Automatic Garbage Collection in Java.

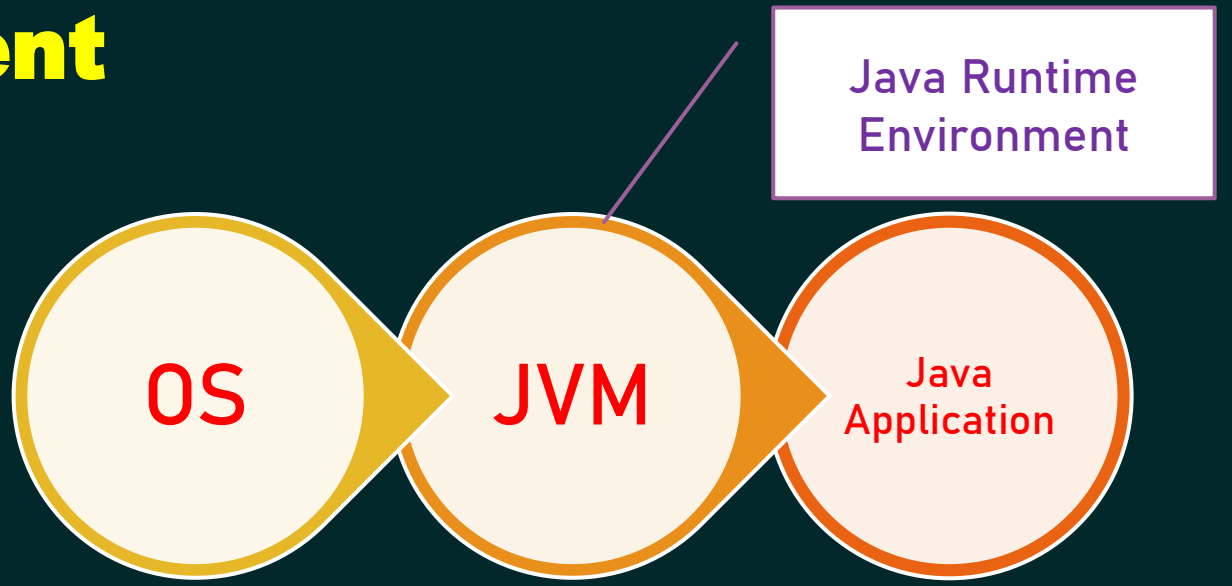
Platform Independent

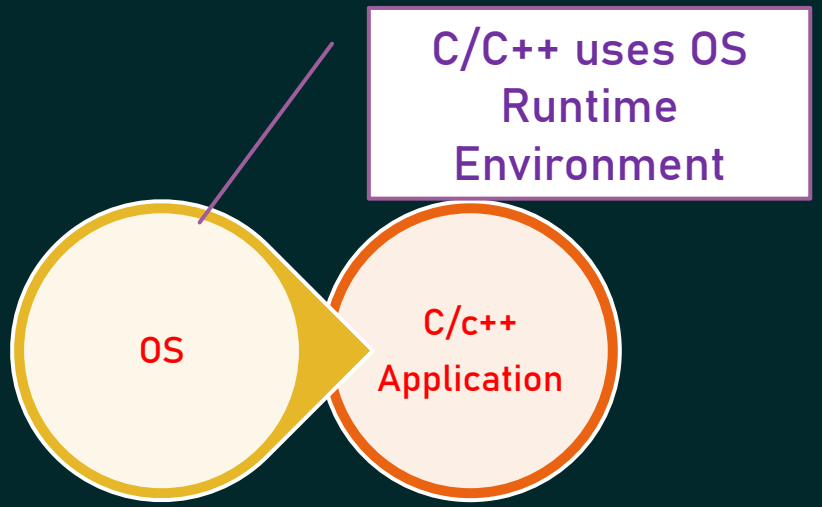
- WORA

WRITE ONCE RUN ANYWHERE

- Supports multiple platforms, for example, Windows, Linux, Sun Solaris, Mac/OS, etc. Compiler compiles and generates bytecode.
- This bytecode is a platform-independent code because it can be run on multiple platforms.

Platform Independent





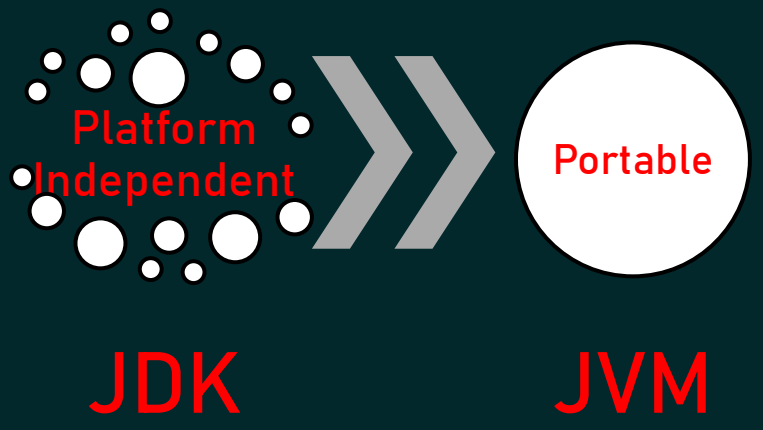
Architecture Neutral

- Java is architecture neutral because there are no implementation dependent features.
- for example, the size of primitive types is fixed.



Portable

- Java an excellent option for cross-platform development.
- Java bytecode facilitates portability.
- It doesn't require any implementation.
- JDK and JVM together makes Java Portable.



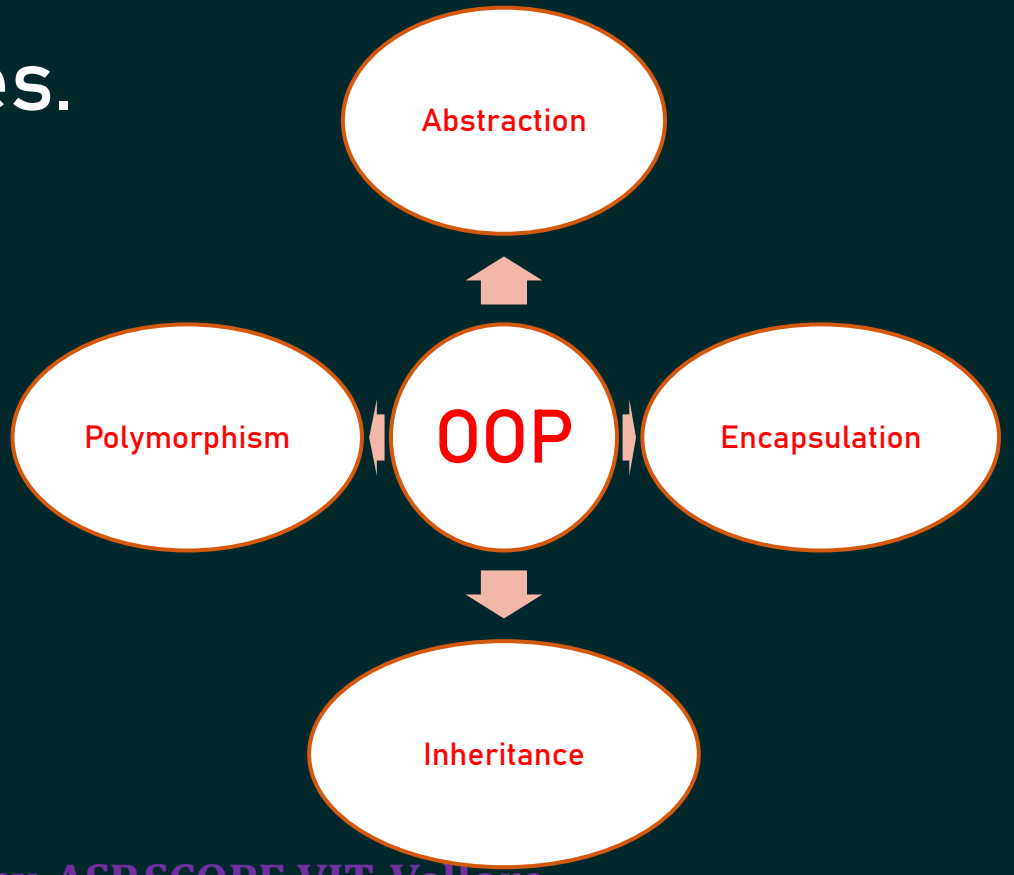
Secure

- Java enables to develop virus-free, tamper-free systems.
- Authentication techniques are based on public-key encryption.
- Provides security using JVM, Sandboxing, Class Loader, Byte code verifier and security manager.



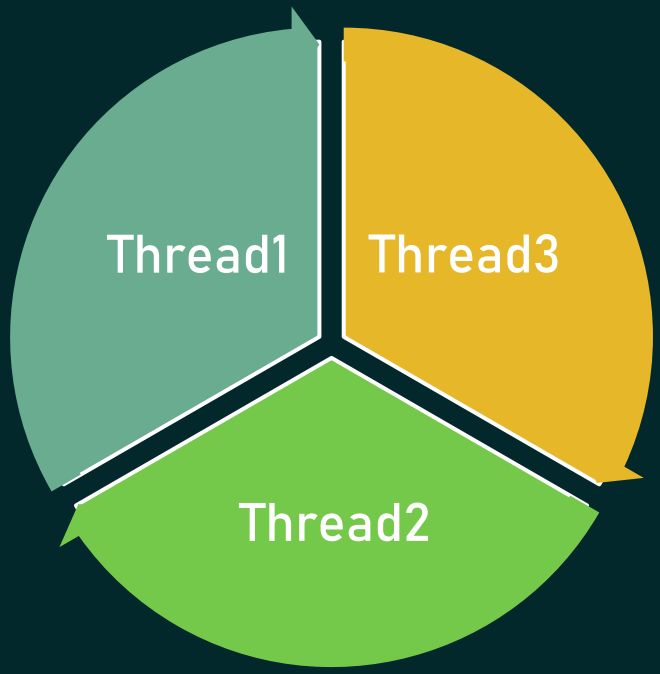
Object Oriented

- It is an Object oriented language.
- Java design every real time problem with class and object.
- Java achieves OOP principles.



Multi threaded

- Simultaneous Execution of a single program.
- Independent execution flow makes threads.
- Threads are lightweight that it doesn't require separate memory space for execution.



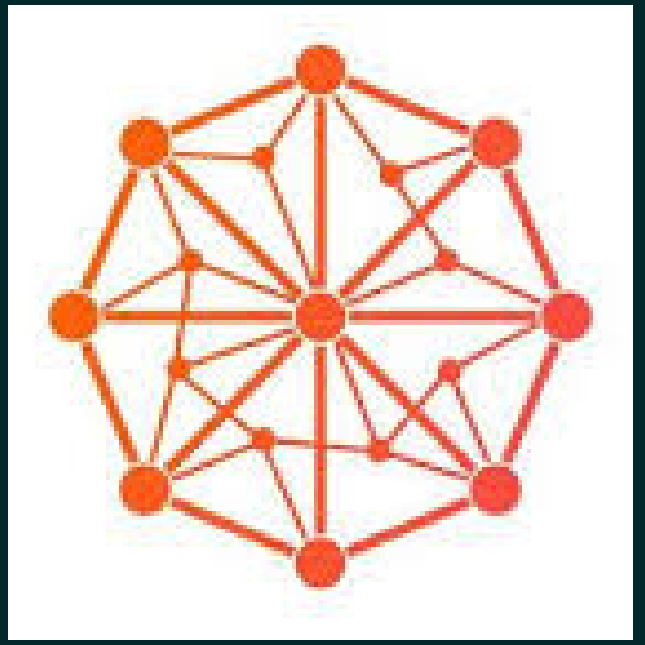
Robust

- Strong Error Handling
- Strongly typed
- Automatic Garbage Collection

Robustness

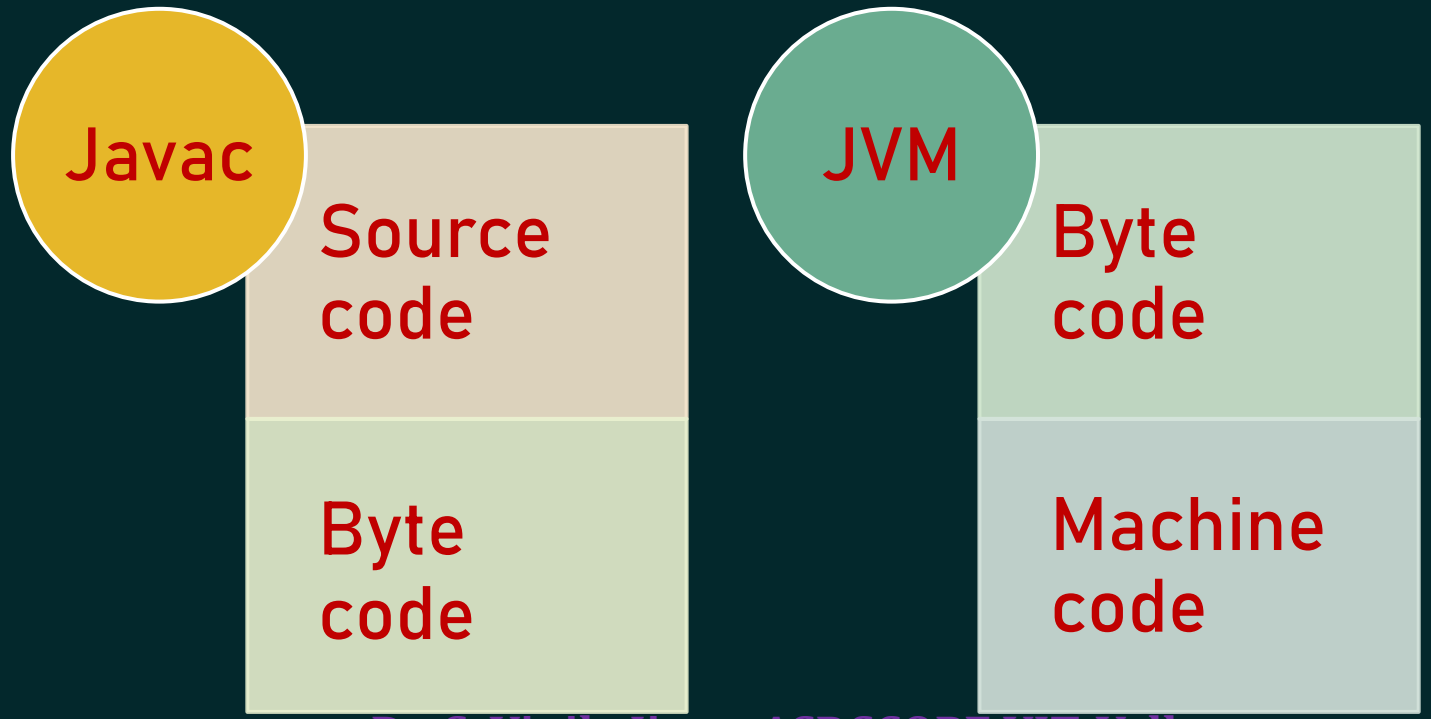
Distributed

- Networking Support
- RMI-Remote Method Invocation
- Web Services
- Object Serialization



Interpreted

- JDK – compiles to byte code
- JVM interpretes byte code
- JIT (Just in time compilation) converts that to native code.



High Performance

- Just in time compilation
- Garbage Collection
- Optimized library support
- Concurrency Support

High Performance

Dynamic

- Dynamic method Invocation
- Reflection
- Dynamic class loading
- Dynamic Linking
- Java Native Interface

Summary

- 12 Features

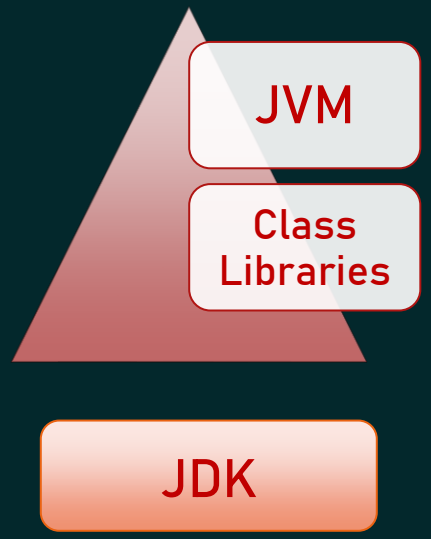
THANK YOU

Java Virtual Machine

- JVM
- Software Installation

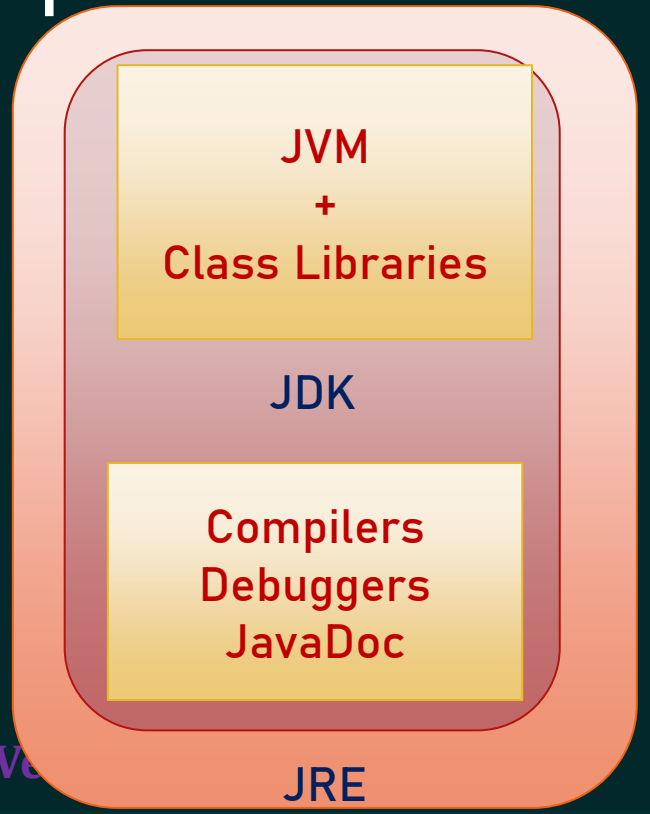
JDK

- JDK (Java Development Kit) is a software development kit required to develop applications in Java.
- JRE is downloaded with jdk.



JRE

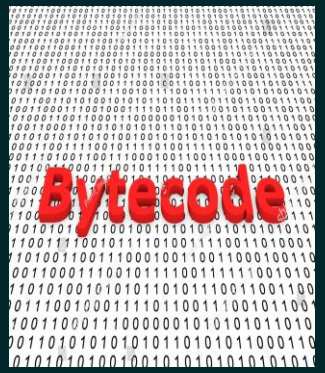
- JRE (Java Runtime Environment) is a software package that provides Java class libraries, Java Virtual Machine (JVM), and other components that are required to run Java applications.
- JRE is the superset of JVM & JDK.



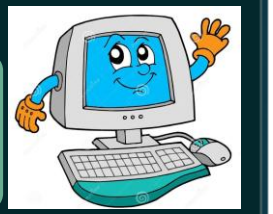
JAVA Program Execution

```
class HelloVIT
{
public static void main(String args[])
{
System.out.println("Hello VIT");
//Prints on screen "Hello VIT"
}
}
```

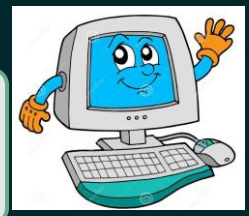




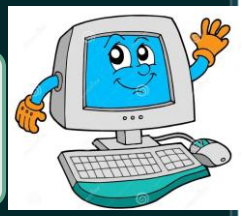
HelloVIT.java → Java Compiler → HelloVIT.class → JVM (Windows) → Machine Code



HelloVIT.class → JVM (Linux) → Machine Code



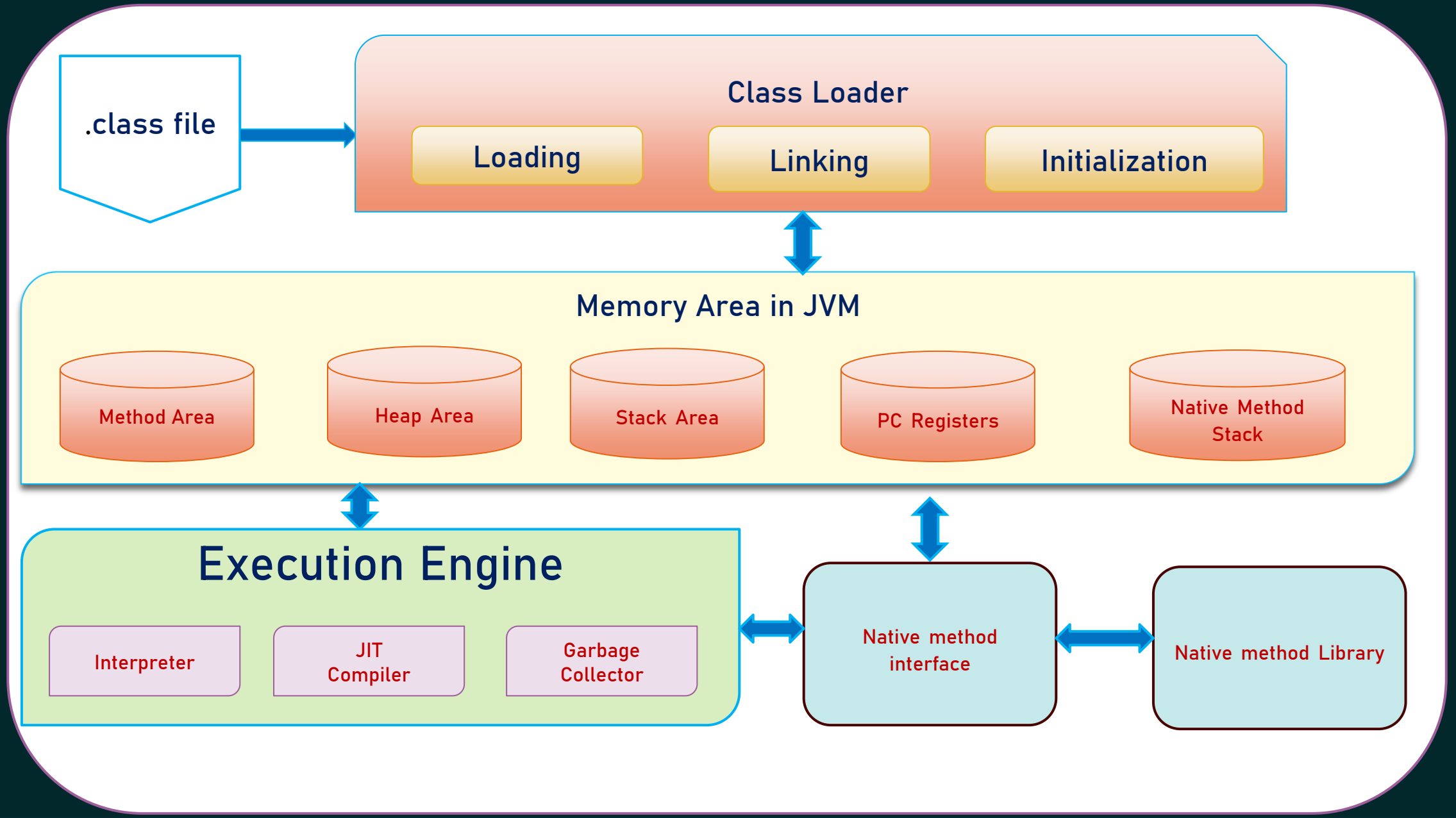
HelloVIT.class → JVM (Macintosh) → Machine Code



JVM

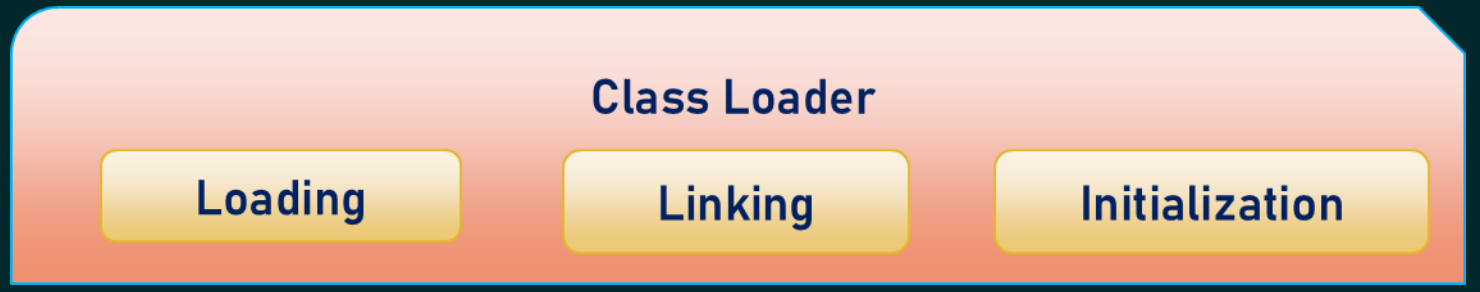
- The Java Virtual Machine is a virtual “machine”.
- It provides a runtime environment for Java applications and programs.
- It interprets and execute Java bytecode.
- Bytecode can be executed by any JVM implementation.

JVM Architecture



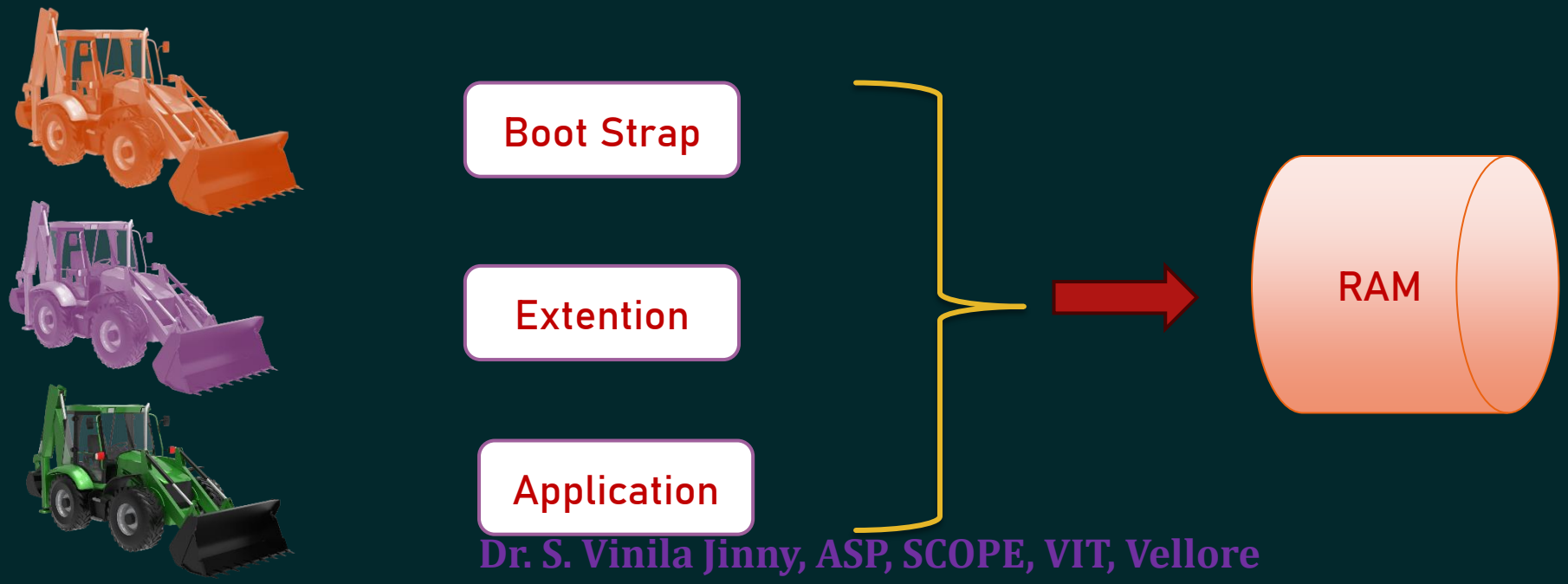
JVM Components – Class Loader

- Class Loader loads the bytecode to the memory
- three phases
  - Loading
  - Linking
  - Initialization



Class Loader - Loading

- 1. Boostrap class Loader – imported class loader
- 2. Extention class Loader – loads additional class
- 3. Application class Loader- our application byte code



Class Loader–Linking

- Verify
- Prepare
- Resolve



Class Loader–Loading, Linking & Initialization

Loading



Linking

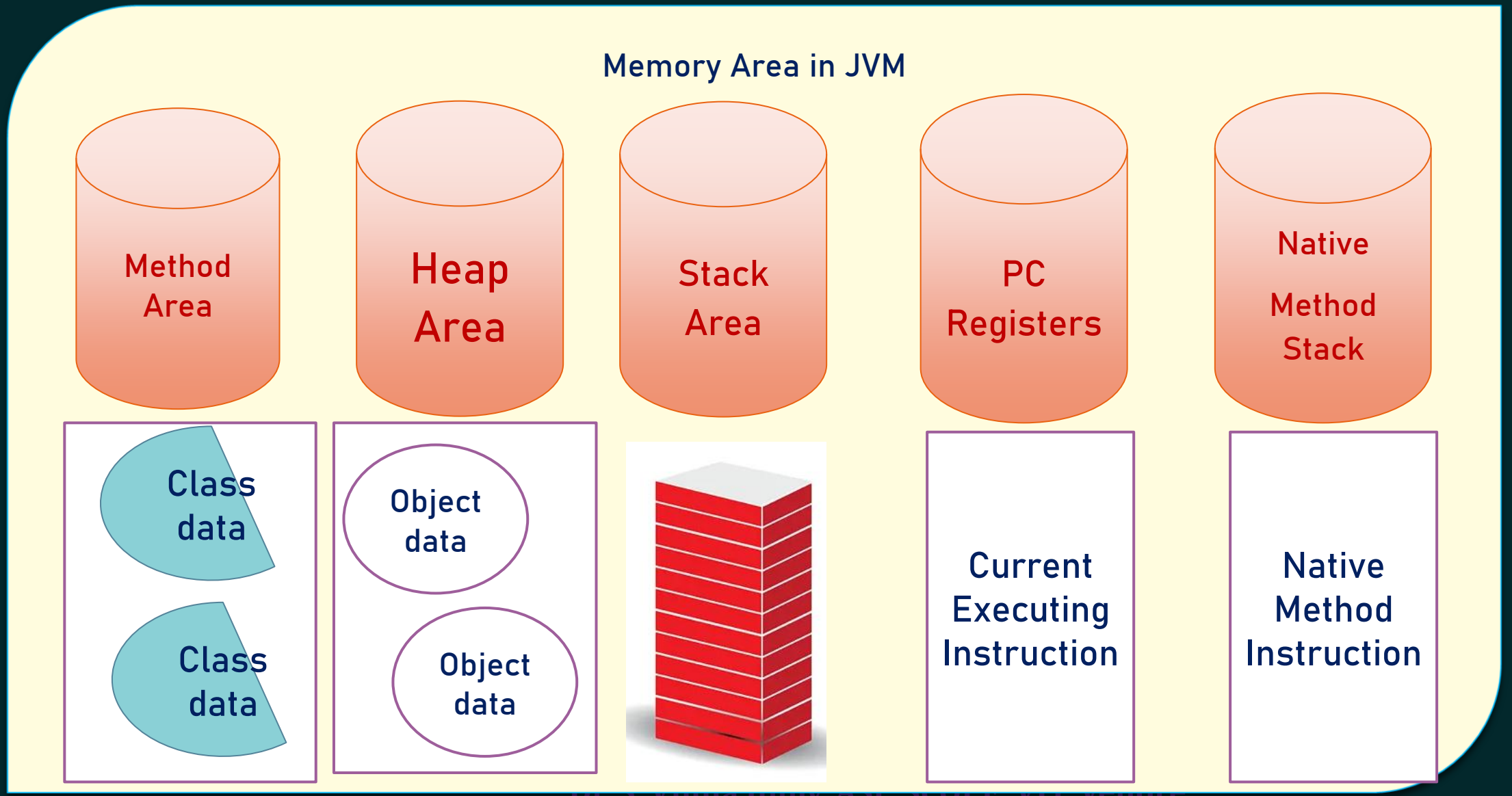
Verify

Prepare

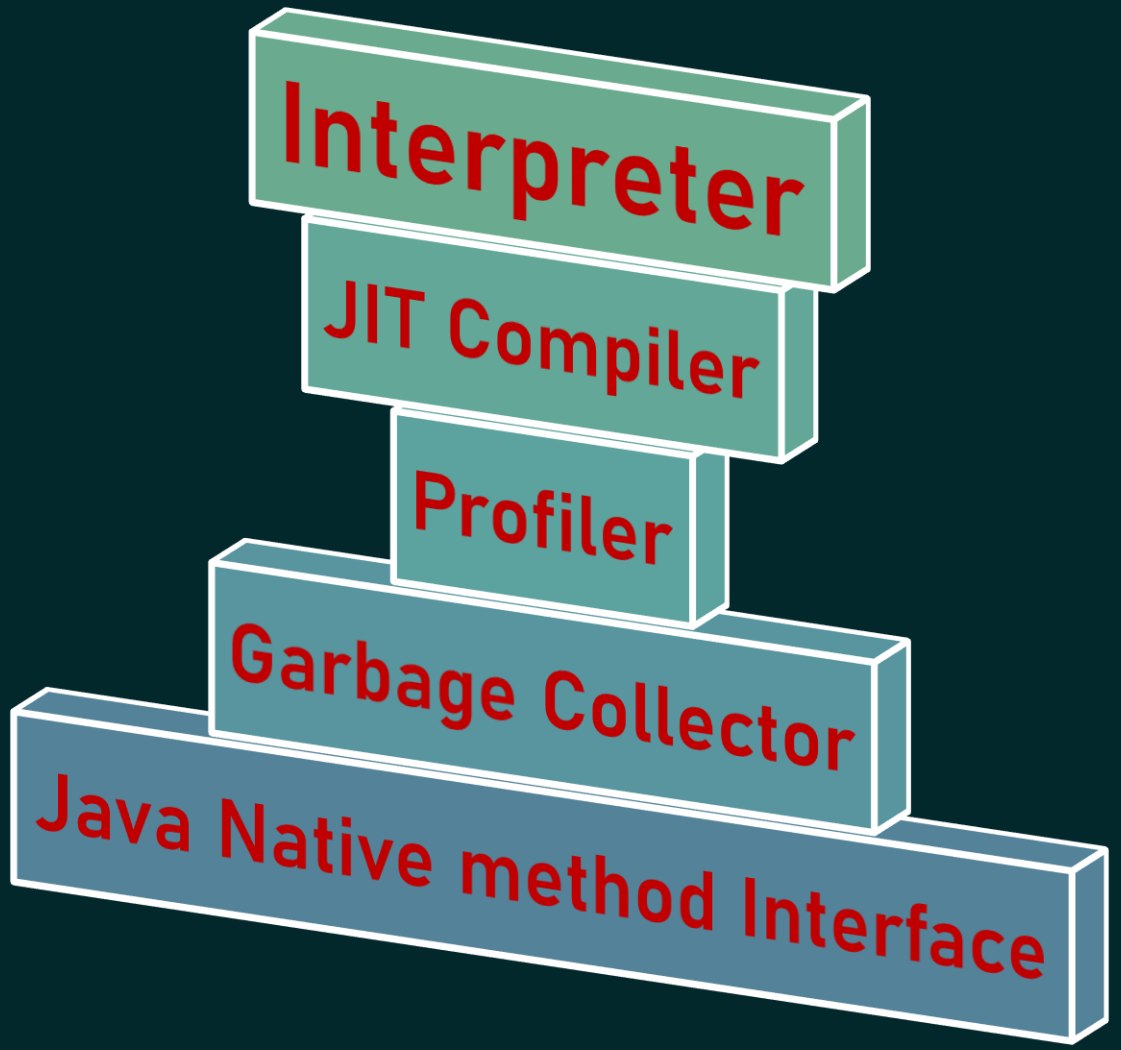
Resolve

Initialization

JVM Component-Runtime Data Area



JVM Component-Execution Engine



Software Installation

- Download Java Software
- Install Java Software
- Verify Installation
- Can Run Java
  - Using Command Prompt and Notepad
  - Using any IDE (Eclipse, NetBeans, IntelliJ…etc..)

Summary

- JVM
- Software Installation

THANK YOU

JAVA Program Structure & Fundamentals

- Java Program Structure
- First Java Program
- **Java Fundamentals**
  - Keywords
  - Identifier
  - Literals
  - Operators
  - Separators

Dr. S. Vinila Jinny, ASP, SCOPE, VIT, Vellore.

Java Program Structure



Java Program Structure

- Documentation Statement → Suggested
- Package Statement
- Import statement
- Interface Statement
- Class Definition

(Package Statement, Import statement, Interface Statement, Class Definition → Optional)

- Main Method Definition → Essential

Dr. S. Vinila Jinny, ASP, SCOPE, VIT, Vellore.



Package Section

- A package is a group of classes that are defined by a name.
- package is a keyword that tells the compiler that package has been created.
- It is declared as:

  package package_name;

- There can be only one package statement in a Java program and it has to be at the beginning of the code before any class or interface declaration.

Java Program Structure

- Documentation Statement → Suggested
- Package Statement
- Import statement
- Interface Statement
- Class Definition

(Package Statement, Import statement, Interface Statement, Class Definition → Optional)

- Main Method Definition → Essential

Dr. S. Vinila Jinny, ASP, SCOPE, VIT, Vellore.



Import Section

- import statement is used to refer to the classes stored in other packages.
- An import statement is always written after the package statement but it has to be before any class declaration.
- We can import a specific class or classes in an import statement.

```
import java.util.Date; //imports the date class
import java.applet.*;  //imports all the classes from the java applet package
```

Java Program Structure

- Documentation Statement → Suggested
- Package Statement
- Import statement
- Interface Statement
- Class Definition

  (Package Statement, Import statement, Interface Statement, Class Definition → Optional)
- Main Method Definition → Essential

Dr. S. Vinila Jinny, ASP, SCOPE, VIT, Vellore.



Interface Section

- An interface is a lot similar to a class in Java but it contains only constants and method declarations.
- An interface cannot be instantiated but it can be implemented by classes or extended by other interfaces.

```
interface stack
{
void push(int item);
void pop();
}
```

Java Program Structure

- Documentation Statement → Suggested
- Package Statement
- Import statement
- Interface Statement
- Class Definition

(Package Statement, Import statement, Interface Statement, Class Definition → Optional)

- Main Method Definition → Essential



Class & Main

- Class

- A Java program may contain several class definitions.
- A class is a collection of variables and methods that operate on the fields.
- Every program in Java will have at least one class with the main method.

- Main Method

- The main method is from where the execution actually starts and follows the order specified for the following statements.

Java Program Structure

- Documentation Statement → Suggested
- Package Statement
- Import statement
- Interface Statement
- Class Definition

(Package Statement, Import statement, Interface Statement, Class Definition → Optional)

- Main Method Definition → Essential



Document Section

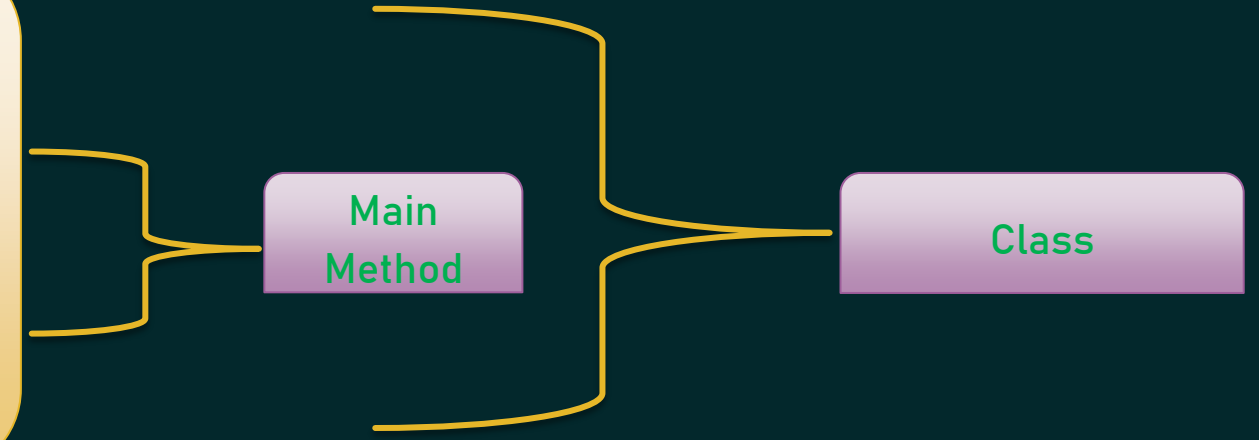
- 3 - types of comments
- Single line Comment
  - // a single line comment is declared like this
- Multi-line Comment
  - /* a multi-line comment is declared like this
    and can have multiple lines as a comment */
- Documentation Comment
  - /** a documentation comment starts with a delimiter and ends with */

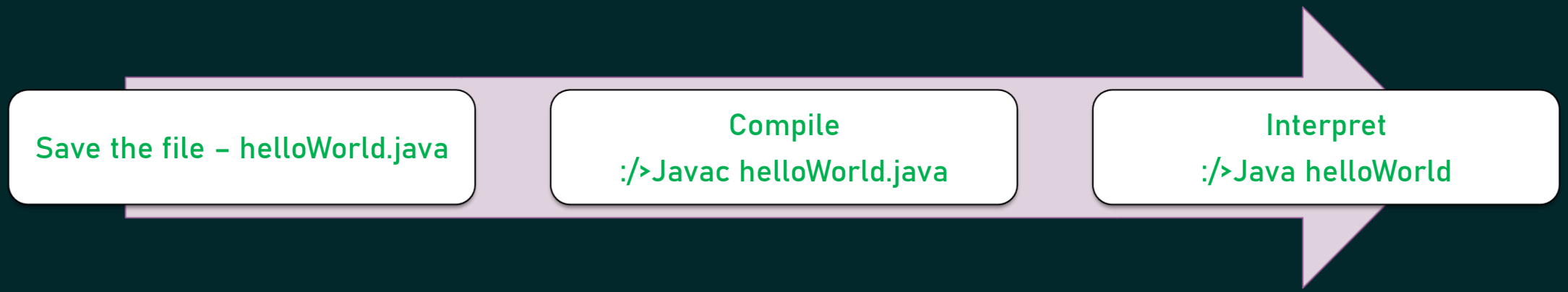
First Java Program

- Type the program in an editor
- Any Text editor can be used for writing Java Program
  - In Windows
    - Notepad, Notepad++, Wordpad, MS-Word etc.,
  - In UNIX
    - Vi, emacs, gedit etc.,
- Save the program with a file name. File name should use the name of the class holding main method. Eg. Hello.java

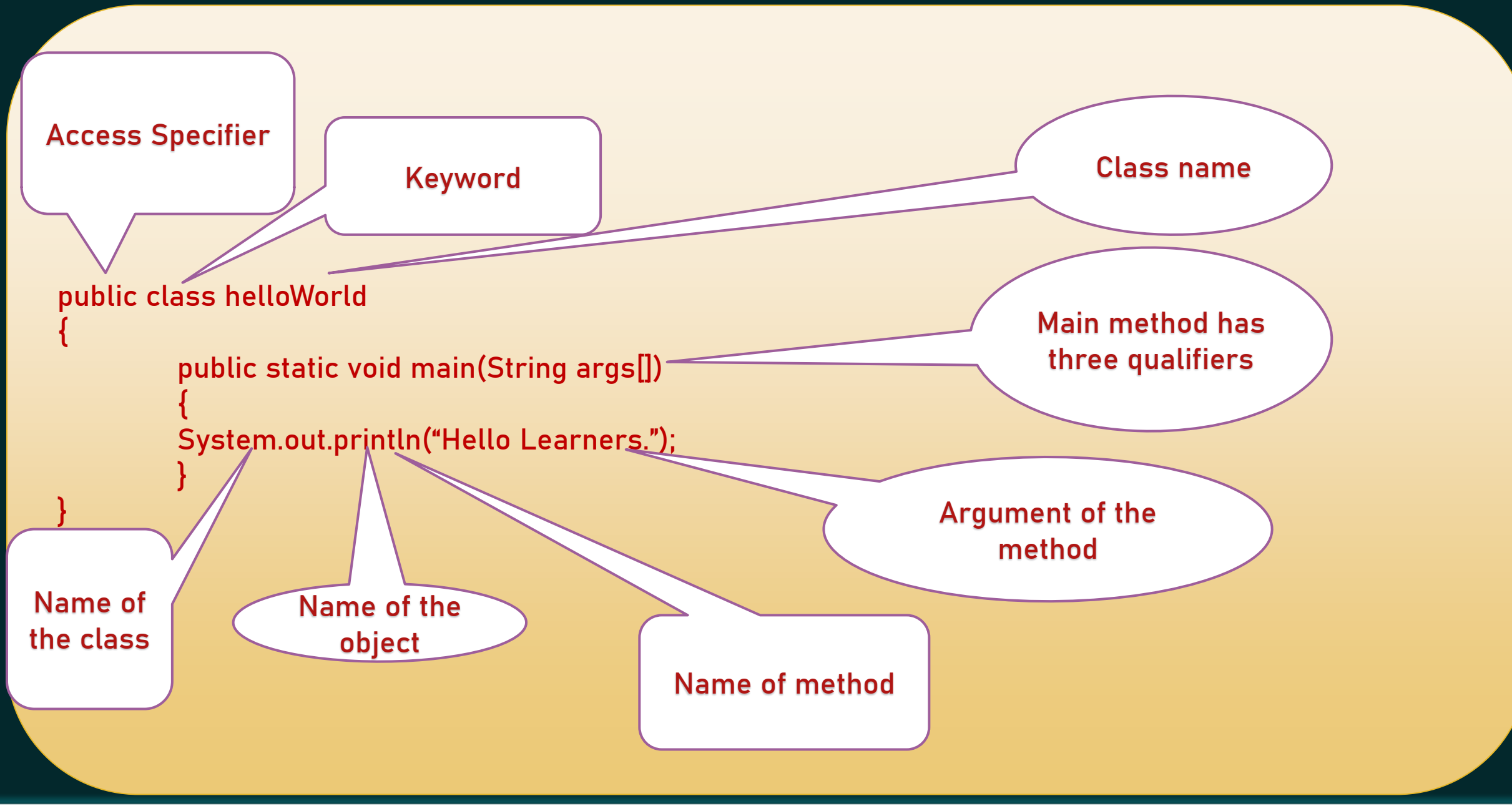
First Java Program

```
public class helloWorld
{
        public static void main(String args[])
        {
        System.out.println("Hello Learners.");
        }
}
```





First Java Program



Java Tokens

- Smallest Individual units in a program
- Identified by the compiler
- Types of tokens
  - Keywords
  - Identifier
  - Literals
  - Operators
  - Separators



Keywords

- Reserved words or predefined words
- Assigned with a special task
- Cannot be used as identifiers
- Always in lowercase

Keywords

01 abstract	02. boolean	03. byte	04. break	05. class
06. case	07. catch	08. char	09. continue	10. default
11. do	12. double	13. else	14. extends	15. final
16. finally	17. float	18. for	19. if	20. implements
21. import	22. instanceof	23. int	24. interface	25. long
26. native	27. new	28. package	29. private	30. protected
31. public	32. return	33. short	34. static	35. super
36. switch	37. synchronized	38. this	39. thro	40. throws
41. transient	42. try	43. void	44. volatile	45. while
46. assert	47. const	48. enum	49. goto	50. strictfp

Identifier

- Identifiers are used to name a variable, constant, function, class, and array.
- Rules:
  - The first letter of an identifier must be a letter, underscore or a dollar sign. It cannot start with digits but may contain digits.
  - The whitespace cannot be included in the identifier.
  - Identifiers are case sensitive.
- Examples
  - Scope
  - Scope_Vit
  - ₹twenty
  - 12hi – invalid
  - _Jinusma
  - VIT
  - A

Literals

Literals are data used for representing fixed values.

- Boolean Literal

  Integer Literal

  Float literal

  char literal

  string literal

- Examples

```
boolean flag1 = false;
int binaryNumber = 0b10010;
int octalNumber = 027;
int decNumber = 34;
int hexNumber = 0x2F;
int binNumber = 0b10010;
double myDouble = 3.4;
float myFloat = 3.4F;
char letter = 'a';
String str1 = "Java Programming";
String str2 = “by James Gosling";
```

Operators

operators are the special symbol that tells the compiler to perform a special operation.

- Arithmetic Operators
- Assignment Operators
- Relational Operators
- Unary Operators
- Logical Operators
- Ternary Operators
- Bitwise Operators
- Shift Operators

Separators

The separators in Java is also known as punctuators.

- <= ; | , | . | (|) | { | } | [|]

Summary

- Java Program Structure
- First Java Program
- **Java Fundamentals**
  - Variables
  - Keywords
  - Datatypes
  - Operators

JAVA Fundamentals

- Java Tokens or Elements
- Variables
- Command Line Arguments

Java Tokens or Elements

- Smallest Individual units in a program
- Identified by the compiler
- Types of tokens
  - Identifier or names
  - Keywords
  - Literals
  - Operators
  - Separators



Identifier

- Identifiers are used to name a variable, constant, function, class, and array.
- Rules:
  - The first letter of an identifier must be a letter, underscore or a currency sign. It cannot start with digits but may contain digits.
  - The whitespace cannot be included in the identifier.
  - Identifiers are case sensitive.
- Examples
  - Scope
  - Scope_Vit
  - ₹rupee
  - $dollar
  - 12hi – invalid
  - _Jinusma
  - VIT
  - A

Keywords

- Reserved words or predefined words
- Assigned with a special task
- Cannot be used as identifiers
- Always in lowercase
- Currently 50 keywords in use.

Keywords

01 abstract	02. boolean	03. byte	04. break	05. class
06. case	07. catch	08. char	09. continue	10. default
11. do	12. double	13. else	14. extends	15. final
16. finally	17. float	18. for	19. if	20. implements
21. import	22. instanceof	23. int	24. interface	25. long
26. native	27. new	28. package	29. private	30. protected
31. public	32. return	33. short	34. static	35. super
36. switch	37. synchronized	38. this	39. throw	40. throws
41. transient	42. try	43. void	44. volatile	45. while
46. assert	47. const	48. enum	49. goto	50. strictfp

Literals

Literals are elements used for representing constant values.

- Boolean Literal
- Integer Literal
  - numeric literals can be declared using underscore characters (_), to improve readability
  - **decimal,binary, octal, hexadecimal**
- Float literal
- Char literal
- String Literal
- Null literal

- Examples

```
boolean flag1 = false;
int with_ = 1_000_000;
int binaryNumber = 0b10010;
int octalNumber = 027;
int decNumber = 34;
int hexNumber = 0x2F;
int binNumber = 0b10010;
double myDouble = 3.4;
float myFloat = 3.4F;
char letter = 'a';
String str1 = "Java Programming";
String str2 = “by James Gosling";
```

Separators

- Separators help us defining the structure of a program
- In Java, there are few characters used as separators

Symbol	Name
( )	Parentheses
{ }	Curly Braces
[ ]	Square Brackets
;	Semicolon
,	Comma
.	Period
:	Colon

Operators

- operators are the special symbol that tells the compiler to perform a special operation on operands.
- Based on the type of operation, it is classified as follows:
  - Arithmetic Operators
  - Assignment Operators
  - Relational Operators
  - Logical Operators
  - Bitwise Operators
- Based on the number of operands, it is classified as follows:
  - Unary Operators
  - Binary Operators
  - Ternary Operators

Arithmetic Operator

Operator	Name	Description	Example
+	Addition	Adds together two values	x + y
-	Subtraction	Subtracts one value from another	x - y
*	Multiplication	Multiplies two values	x * y
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	x % y
++	Increment	Increases the value of a variable by 1	++x
--	Decrement	Decreases the value of a variable by 1	--x

Assignment Operator

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
|=	x |= 3	x = x | 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

Relational Operator

Operator	Description	Example
==	Is Equal To	3 == 5 returns false
!=	Not Equal To	3 != 5 returns true
>	Greater Than	3 > 5 returns false
<	Less Than	3 < 5 returns true
>=	Greater Than or Equal To	3 >= 5 returns false
<=	Less Than or Equal To	3 <= 5 returns true

Logical Operator

Operator	Name	Description	Example
&&	Logical and	Returns true if both statements are true	x < 5 && x < 10
||	Logical or	Returns true if one of the statements is true	x < 5 || x < 4
!	Logical not	Reverse the result, returns false if the result is true	!(x < 5 && x < 10)

Bitwise Operator

Operator	Description
~	Bitwise Complement
<<	Left Shift
>>	Right Shift
>>>	Unsigned Right Shift
&	Bitwise AND
^	Bitwise exclusive OR

Unary Operator

Operator	Meaning
+	Unary plus: not necessary to use since numbers are positive without using it
-	Unary minus: inverts the sign of an expression
++	Increment operator: increments value by 1
--	Decrement operator: decrements value by 1
!	Logical complement operator: inverts the value of a boolean

Binary Operator

- All operators which processes two operands are Binary operators.

```
public class ArithmeticOperator
{
    public static void main (String[]args)
    {
        int A = 10;
        int B = 20;
        System.out.println (A + B);
        System.out.println (A - B);
        System.out.println (A * B);
        System.out.println (A / B);
        System.out.println (A % B);
    }
}
```

Ternary Operator

Syntax:

```
variable = Expression ? expression1 : expression2
```

```
class ternaryexample
{
  public static void main(String[] args)
  {
    int februaryDays = 29;
    String result;
    result = (februaryDays == 28) ? "Not a leap year" : "Leap year";
    System.out.println(result);
  }
}
```

Instance of Operator

- checks whether an object is an instanceof a particular class

```
class inopexample
{
  public static void main(String[] args)
  {
    String str = “VITOL";
    boolean result;
    result = str instanceof String;
    System.out.println("Is str an object of String? " + result);
  }
}
```

JAVA Datatypes

- Type of data that the variable may be assigned
- Two types
  - Primitive datatypes – simple and predefined types given by the language
  - Non-primitive datatypes – derived types classes, arrays, interfaces.

JAVA Primitive Datatypes

Data Type	Size	Description
byte	1 byte	Stores whole numbers from -128 to 127
short	2 bytes	Stores whole numbers from -32,768 to 32,767
int	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
long	8 bytes	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4 bytes	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
double	8 bytes	Stores fractional numbers. Sufficient for storing 15 decimal digits
boolean	1 bit	Stores true or false values
char	2 bytes	Stores a single character/letter or ASCII values

JAVA Variables

- Variables are containers for storing data values.
- Can initialize with values
- Types of Variables in Java
  - Local Variables
  - Instance Variables
  - Static Variables

Syntax:
Datatype identifier;

- Example:

```
int x,y,z;
int x=10,y=100;
int z=x;
```

Local variables

- A variable defined within a block or method or constructor is called a local variable.
- The Local variable is created at the time of declaration and destroyed after exiting from the block or when the call returns from the function.
- The scope of these variables exists only within the block in which the variables are declared, i.e., we can access these variables only within that block.
- Initialization of the local variable is mandatory before using it in the defined scope.

Instance variables

- Instance variables are non-static variables and are declared in a class outside of any method, constructor, or block.
- As instance variables are declared in a class, these variables are created when an object of the class is created and destroyed when the object is destroyed.
- Initialization of an instance variable is not mandatory.
- Instance variables can be accessed only by creating objects.

Static variables

- Static variables are also known as class variables.
- Declared using the static keyword within a class outside of any method, constructor, or block.
- Static variables are created at the start of program execution and destroyed automatically when execution ends.

Command Line Arguments

All command-line arguments are treated as String objects, even if they represent another data type.

```
public class Zoo {

public static void main(String[] args)
    {
        System.out.println(args[0]);
        System.out.println(args[1]);
    }
}
```

```
> javac Zoo.java
> java Zoo Bronx Zoo
>  java Zoo "San Diego" Zoo
>  java Zoo
```

Importing

```
import java.util.Random; // import tells us where to find Random
public class ImportExample {
public static void main(String[] args) {
Random r = new Random();
System.out.println(r.nextInt(10)); // print a number between 0 and 9
}
}
```

com.amazon.java8book – tells us the code came from amazon.com.

Wildcards

```
import java.util.*; // imports java.util.Random among other things
public class ImportExample {
public static void main(String[] args) {
Random r = new Random();
System.out.println(r.nextInt(10));
}
}
```

- The * is a wildcard that matches all classes in the package.

Redundant imports

```
1: import java.lang.System;
2: import java.lang.*;
3: import java.util.Random;
4: import java.util.*;
5: public class ImportExample {
6: public static void main(String[] args) {
7: Random r = new Random();
8: System.out.println(r.nextInt(10));
9: }
10: }
```

Find out the redundant statements?

Summary

- Java Tokens
- Variables
- Command Line Arguments

JAVA Control Statements

- Control Statement
- Types of Control Statement
- Selection Statement

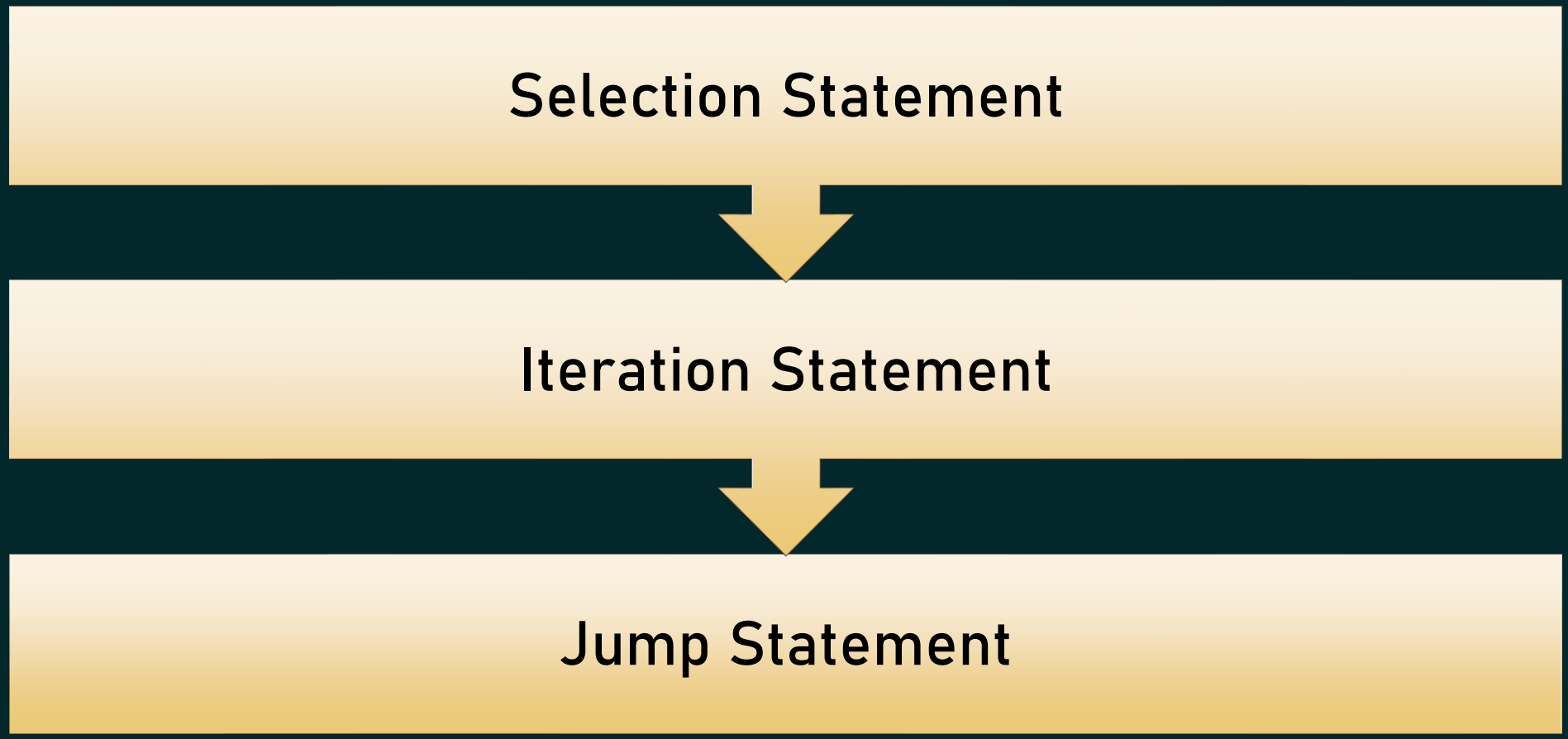
Control Statement

- It controls the flow of execution of the program.
- It uses condition to decide the flow of execution.



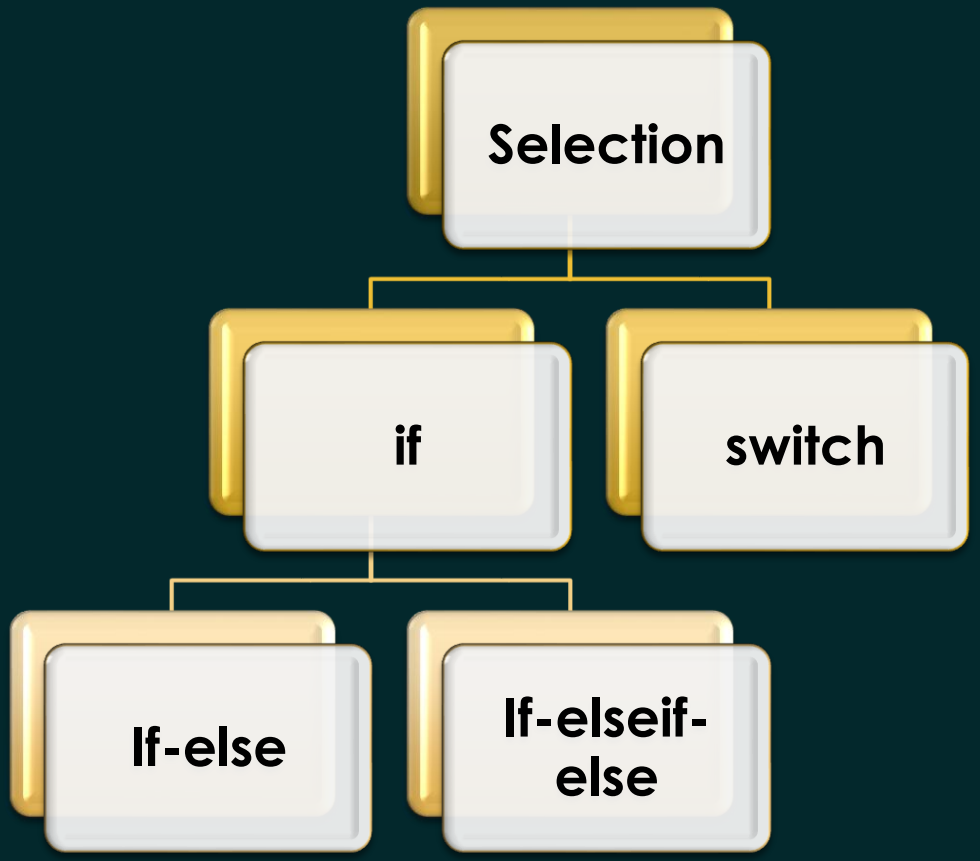


Types of Control Statement



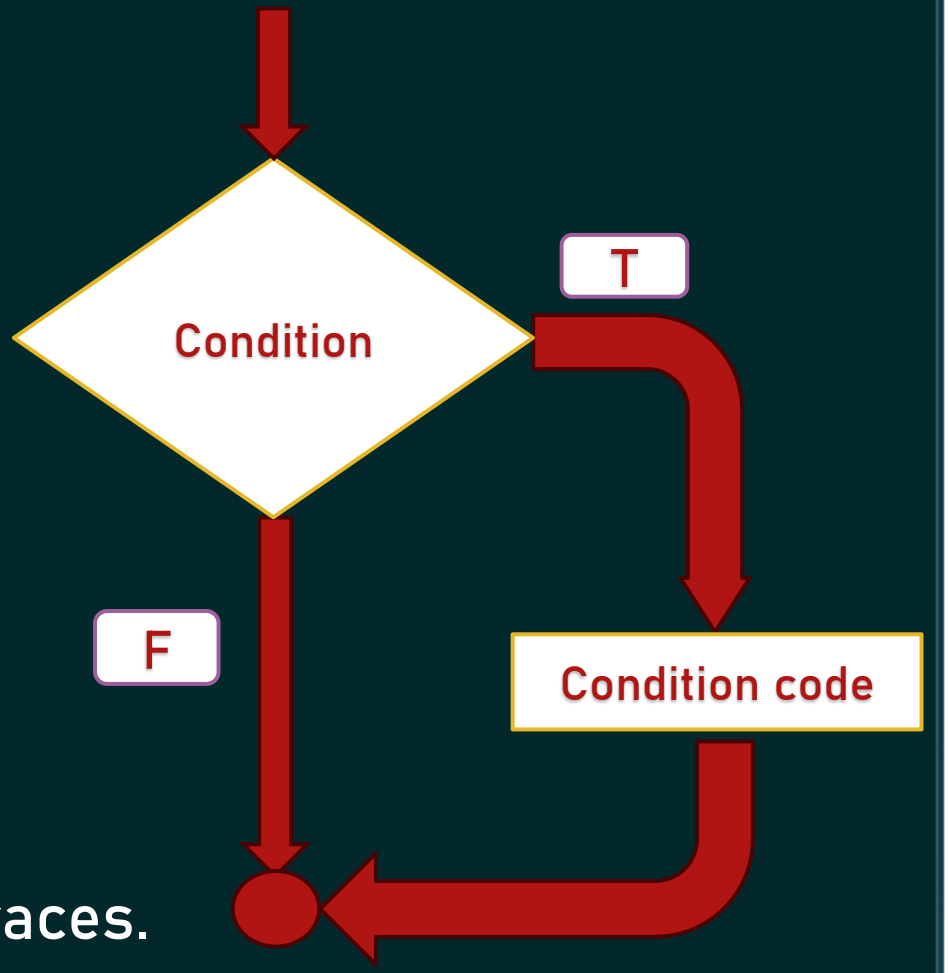
Selection Satement

- Allow the program to choose different paths of execution based upon the outcome of an expression or the state of a variable.



if statement

Syntax:
```
if (condition) statement1;
statement2;
```

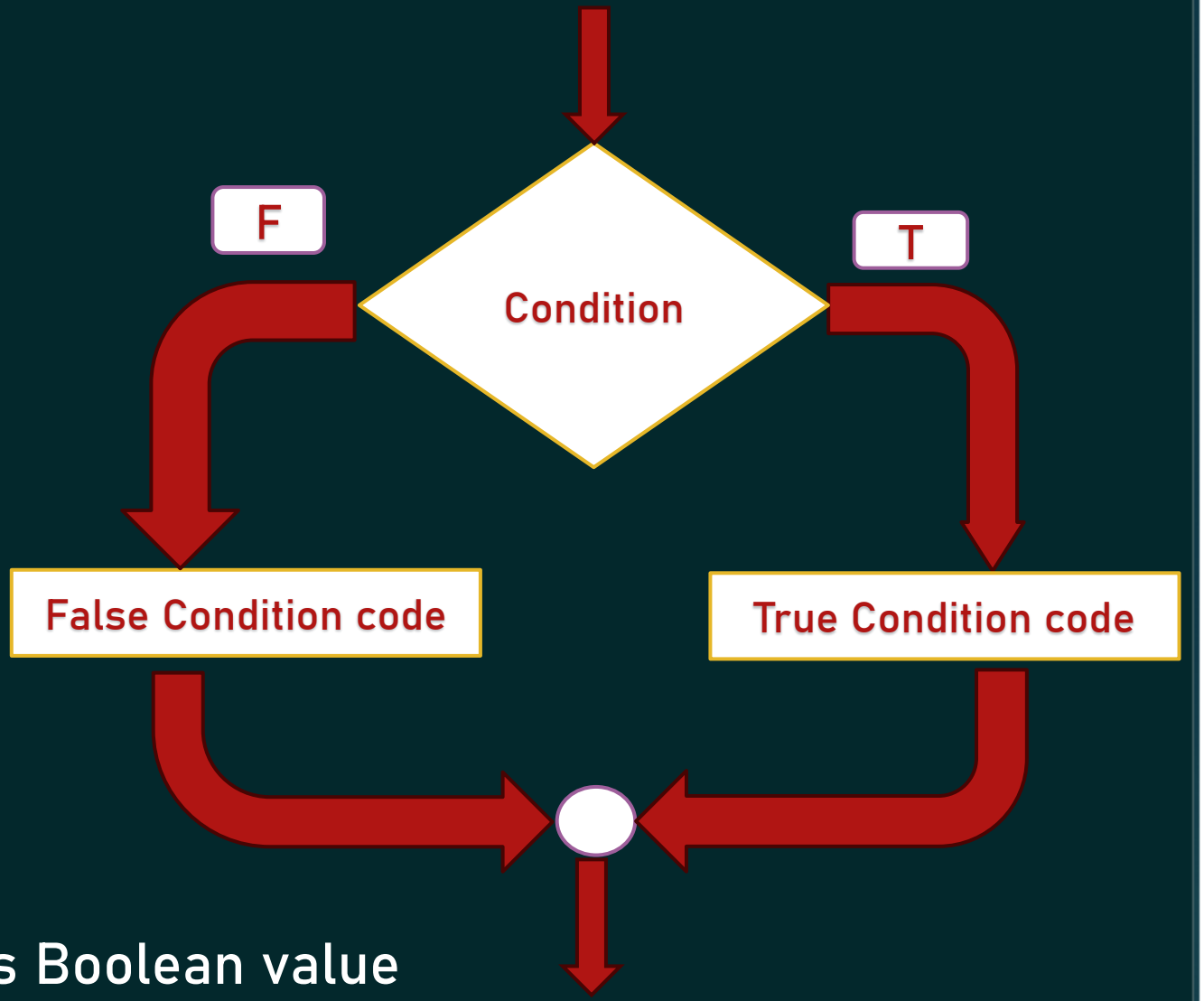


- Statement1 can be single or block of statements
- Block of statements should be placed in curly braces.

if – else statement

Syntax:

```
if (condition) statement1;
else
statement2;
```



- Condition in any expression returns Boolean value
- No case will both statements be executed.

Nested if statement

Syntax:

```
if (condition)
{
statement1;
        if (condition)
        {
        statement
        }
}
```

Multiple Condition Checking

- The expression used to control the if will involve the relational operators.
- It is possible to control the if using a single boolean variable.

If-elseif –else statement

Syntax:

```
if (condition)
{
statement1;
else if (condition)
{
        statement2;
}
else if (condition)
{
        statement3;

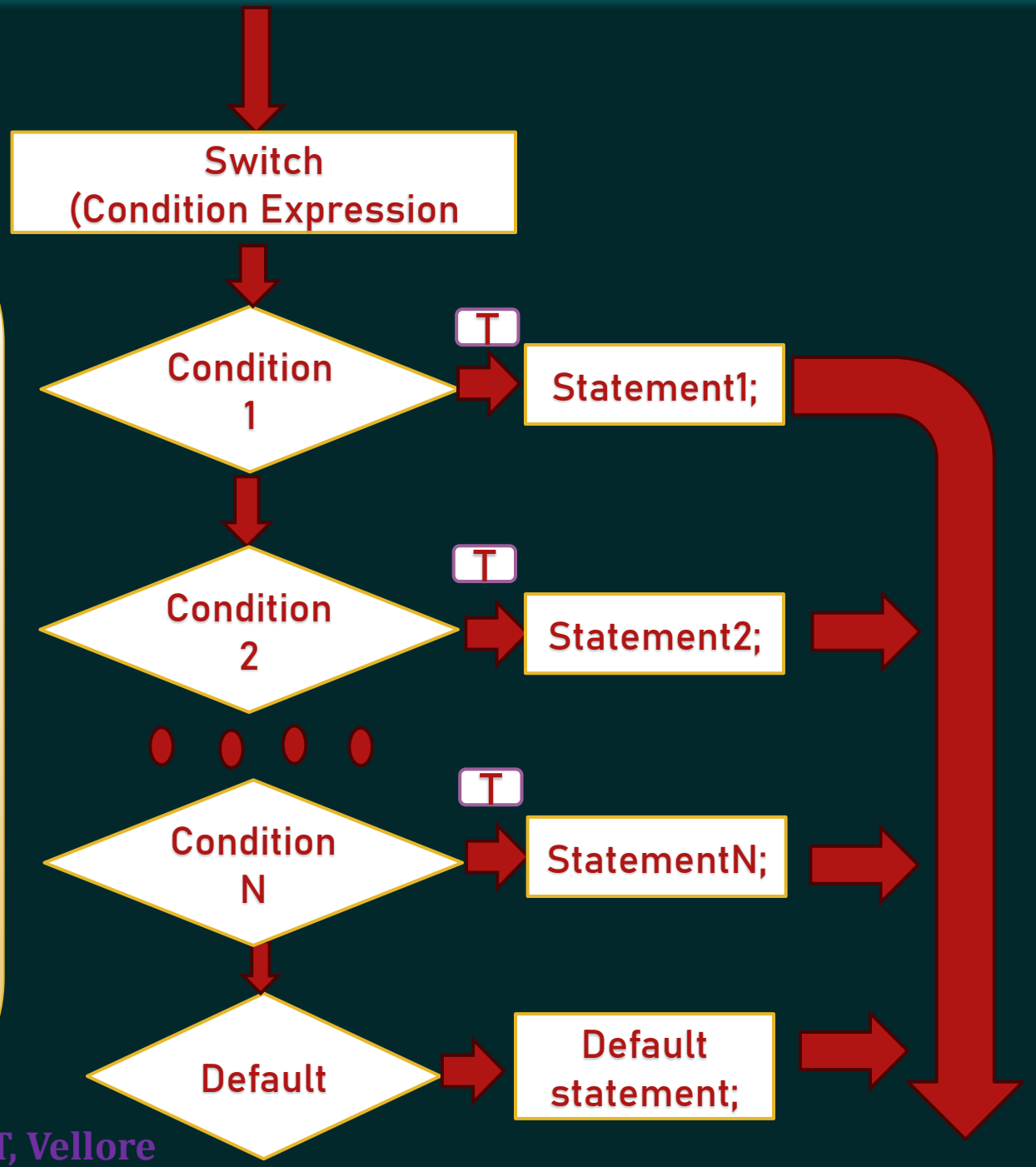
}
else
{
        statement4;
}
```

Multiple Condition Checking

switch statement

Syntax:

```
switch (expression)
{
        case value1:
                //statement sequence;
        case value2:
                //statement sequence;
.....
        case valueN:
                //statement sequence;
        default:
                //default sequence;
}
```



Nested Switch statement

Syntax:

```
switch (expression)
{
        case value1:
                switch (expression)
                {
                        case value1:
                                statement sequence;
                        case value2:
                        ……
                        default:
                                statement sequence;
                }
        ….
        case valueN:
                //statement sequence;
        default:
                //default sequence;
}
```

A switch statement is usually more efficient than a set of nested ifs.

Example-1

- Problem Statement : Check whether the given student is pass .

```
public class GradeCheck
{
    public static void main(String[] args)
      {
            int grade = 65;                    // Student's grade
            if (grade  > = 50)
            {
               System.out.println("Congratulations! You have passed.");
            }
    }
}
```

Output
Congratulations! You have passed.

Example-2

- Problem Statement : Check whether the given year is leap year .

```
public class LeapYearCheck
{
    public static void main(String[] args)
      {
              int year = 2020;      // Year to check
               if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))
              {
               System.out.println(year + " is a leap year.");
               }
       }
}
```

Output
2020 is a leap year.

Example-3

- Problem Statement : Check whether the given number is odd or not.

```
public class EvenOddChecker {
    public static void main(String[] args) {
        int number = 10;
       if (number % 2 == 0)
       {
            System.out.println(number + " is even");
        }
       else
       {
            System.out.println(number + " is odd");
        }
    }
}
```

Output

10 is even

Example-4

- Problem Statement : Check whether the given number is which day.

```
int num = 3;
switch (num) {
  case 1:
    System.out.println("One");
  case 2:
    System.out.println("Two");
  case 3:
    System.out.println("Three");
  case 4:
    System.out.println("Four");
  case 5:
    System.out.println("Five");
  default:
    System.out.println("Invalid number");
}
```

Output

Three
Four
Five
Invalid number

Summary

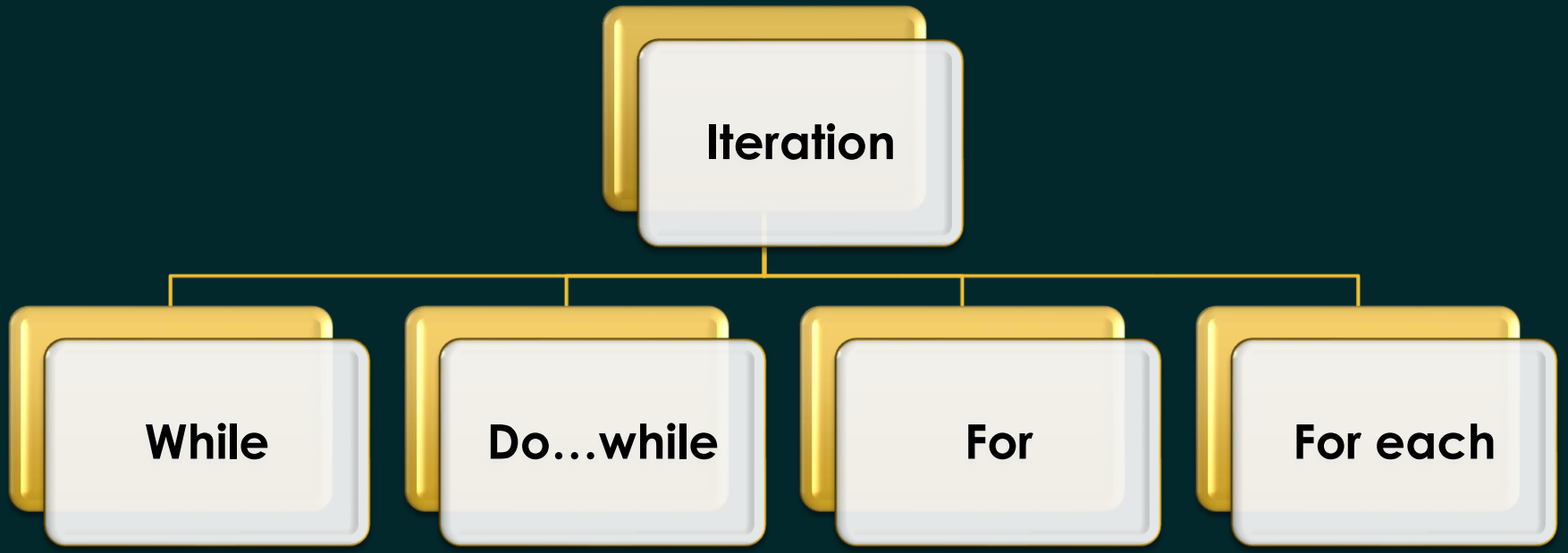
- Control Statement
- Types of Control Statement
- Selection Statement
- Example programs

JAVA Control Statements

- Iteration Statements
- Jump Statements

Iteration Statement

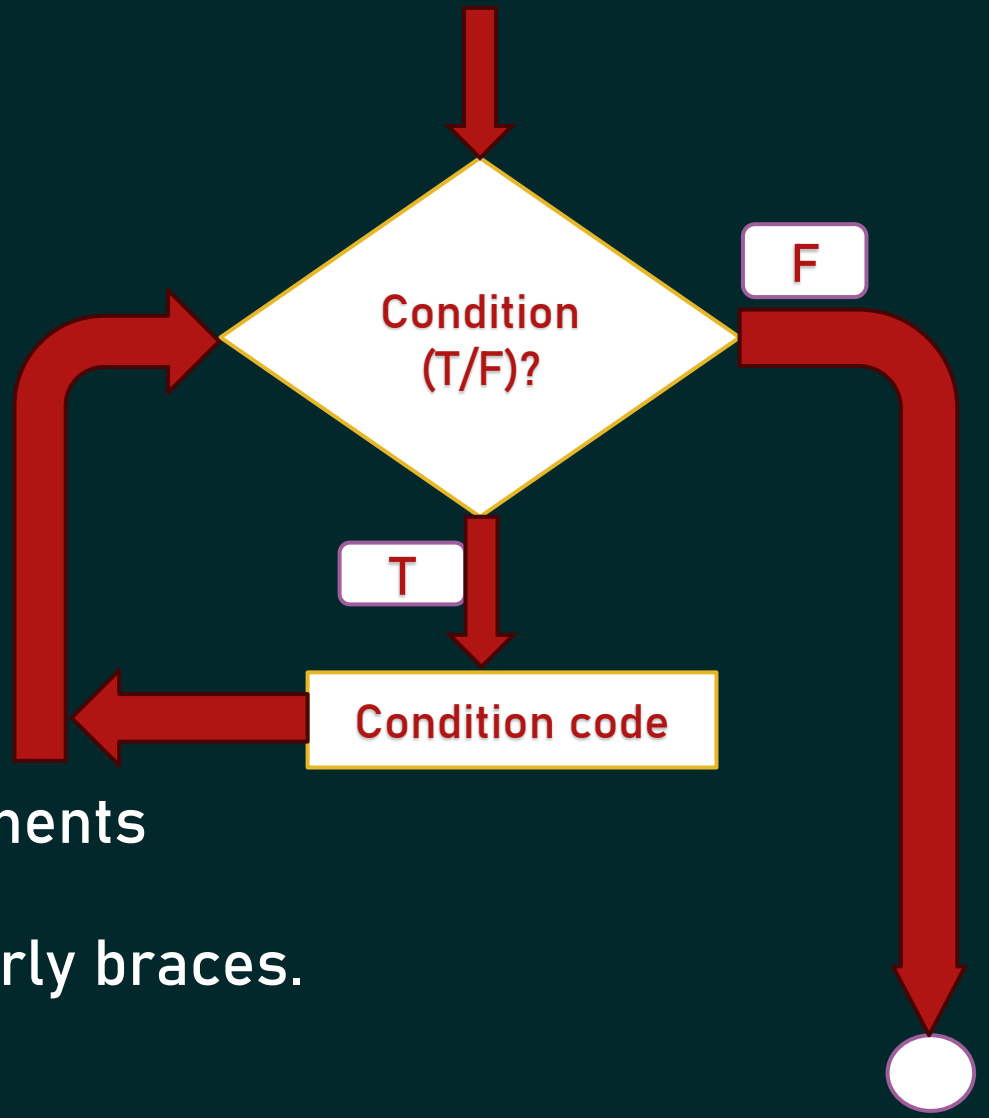
- Allow the program to execute a part of the program repeatedly.
- Called as looping or iterative statements.



while statement

Syntax:

```
while(condition) statement1;
statement2;
```



- Statement1 can be single or block of statements
- Block of statements should be placed in curly braces.

Example-5

- Problem Statement : Find the factorial of a given number

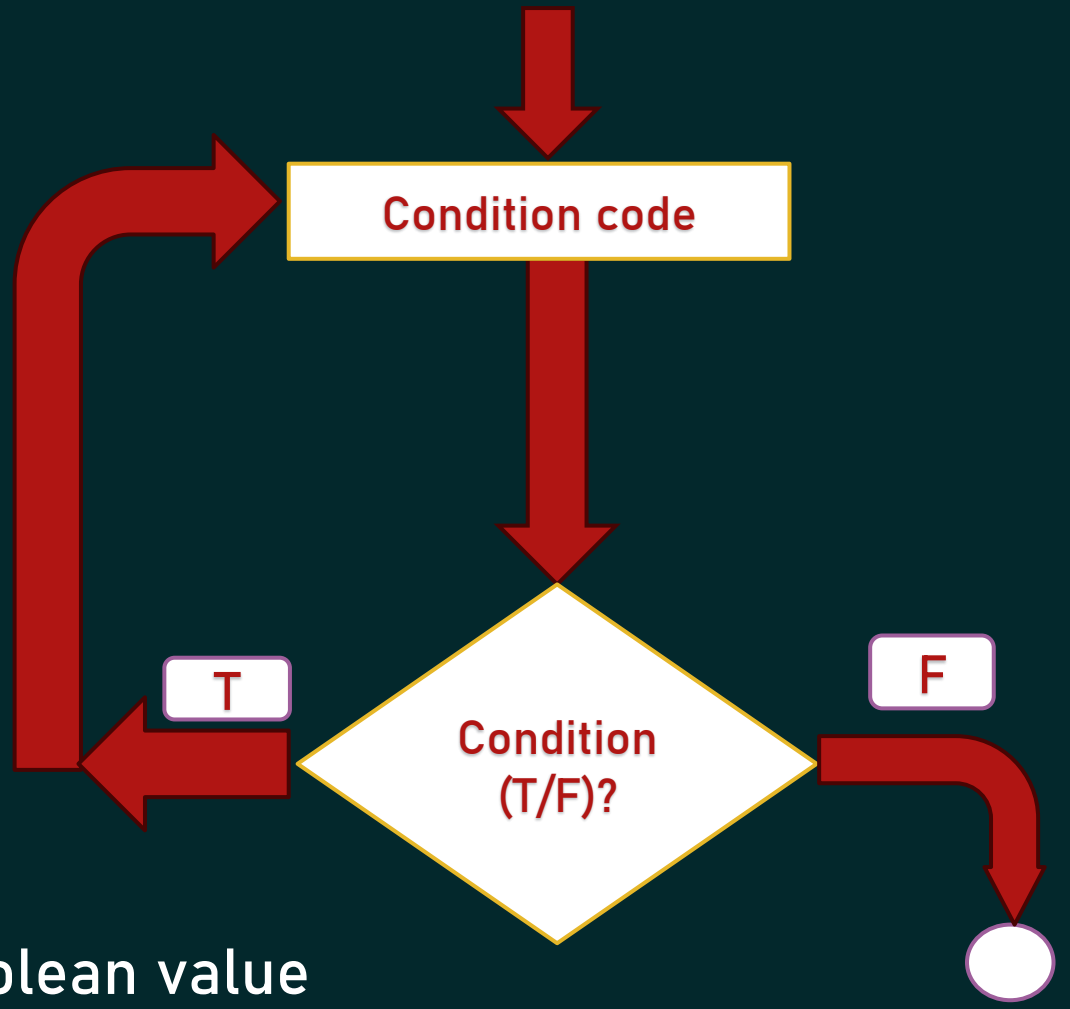
```
public class WhileLoopExample {
    public static void main(String[] args) {
        int num = 5;
        int factorial = 1;
        int i = 1;
        while (i <= num) {
                        factorial *= i;
                         i++;
                    }
                System.out.println("Factorial of " + num + " is: " + factorial);
    }
}
```

Output
Factorial of 5 is: 120

Do...while statement

Syntax:

```
do
{
statement1;
}while(condition);
statement2;
```



- Condition in any expression returns Boolean value

Example-6

- Problem Statement : Sum of N numbers

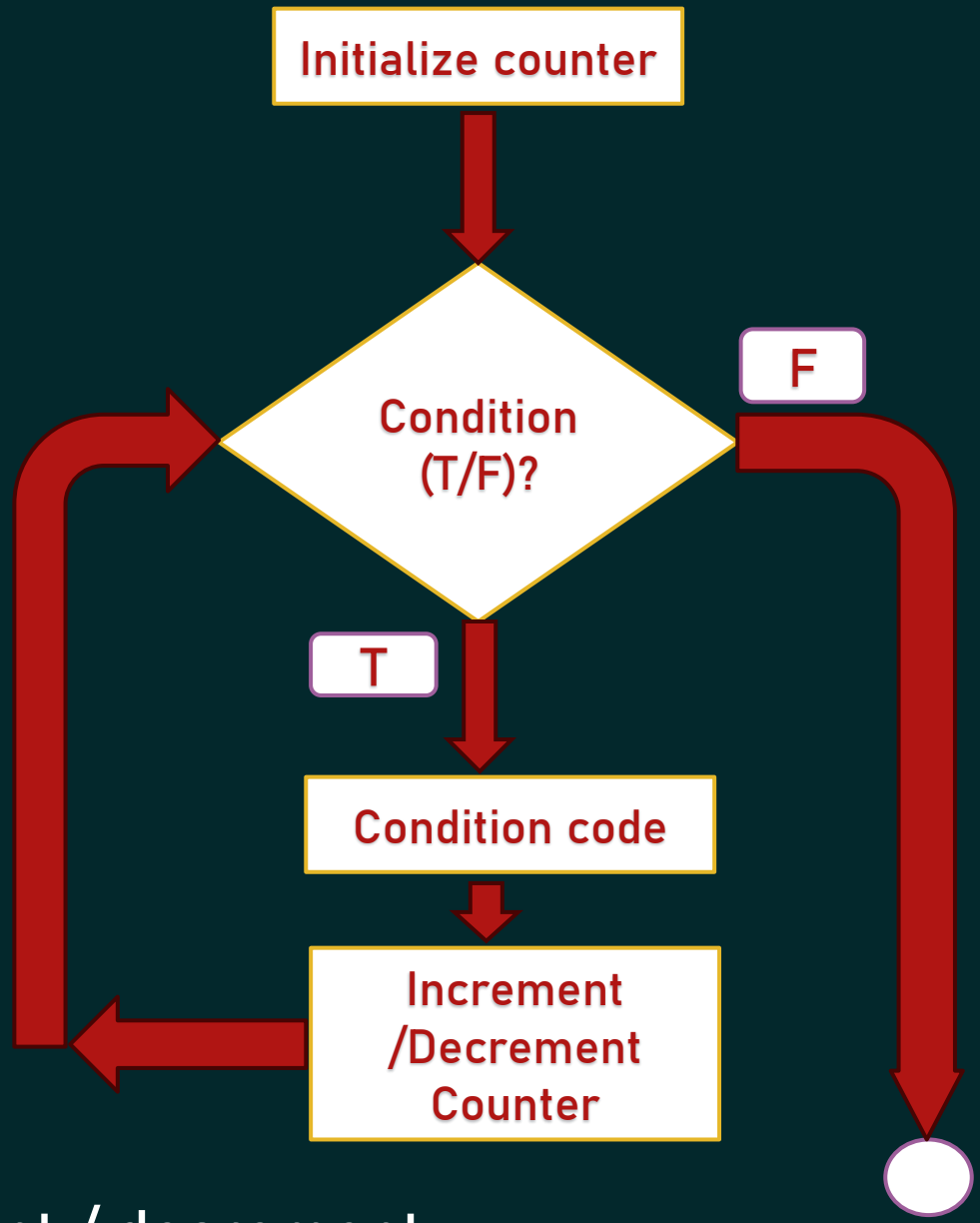
```
public class DoWhileExample {
    public static void main(String[] args) {
        int i = 1;
        int sum = 0;
            do {
                sum += i;
                i++;
              } while (i <= 5);
            System.out.println("Sum of numbers from 1 to 5 is: " + sum);
    }
}
```

Output
Sum of numbers from 1 to 5 is: 15

for statement

Syntax:

```
for( initialization ; condition; increment/decrement)
        { statement1; }
statement2;
```



- Includes three components
- Intialization, condition checking and increment / decrement

Example-7

- Problem Statement : Print even numbers between 1-10

```
class ForLoopExample
{
public static void main(String[] args)
  {
   for (int i=1; i<=10; i++)
    {
     if (i%2==0)
     System.out.print(i);
    }
     System.out.println("Loop Ending");

  }
}
```

Output
2 4 6 8 10
Loop Ending

for statement

For Statement allows following variations

- Declaring Loop Control Variables Inside the for Loop

```
for (int i=1; i<=10; i++)
```

- Using the Comma for multiple initialization and counter variables

```
for (int i=1,j=10; i<=10 && j>0; i++,j--)
```

for statement

For Statement allows following variations

- Condition statement can be any Boolean expression

```
boolean done=false;
for (int i=1; !done; i++)
```

- Missing parts of for loop

```
int i=1
for ( ; i<=10; ){
        i++
}
```

```
for ( ;; )
    {
//code
    }
```

```
for ( ;; );
```

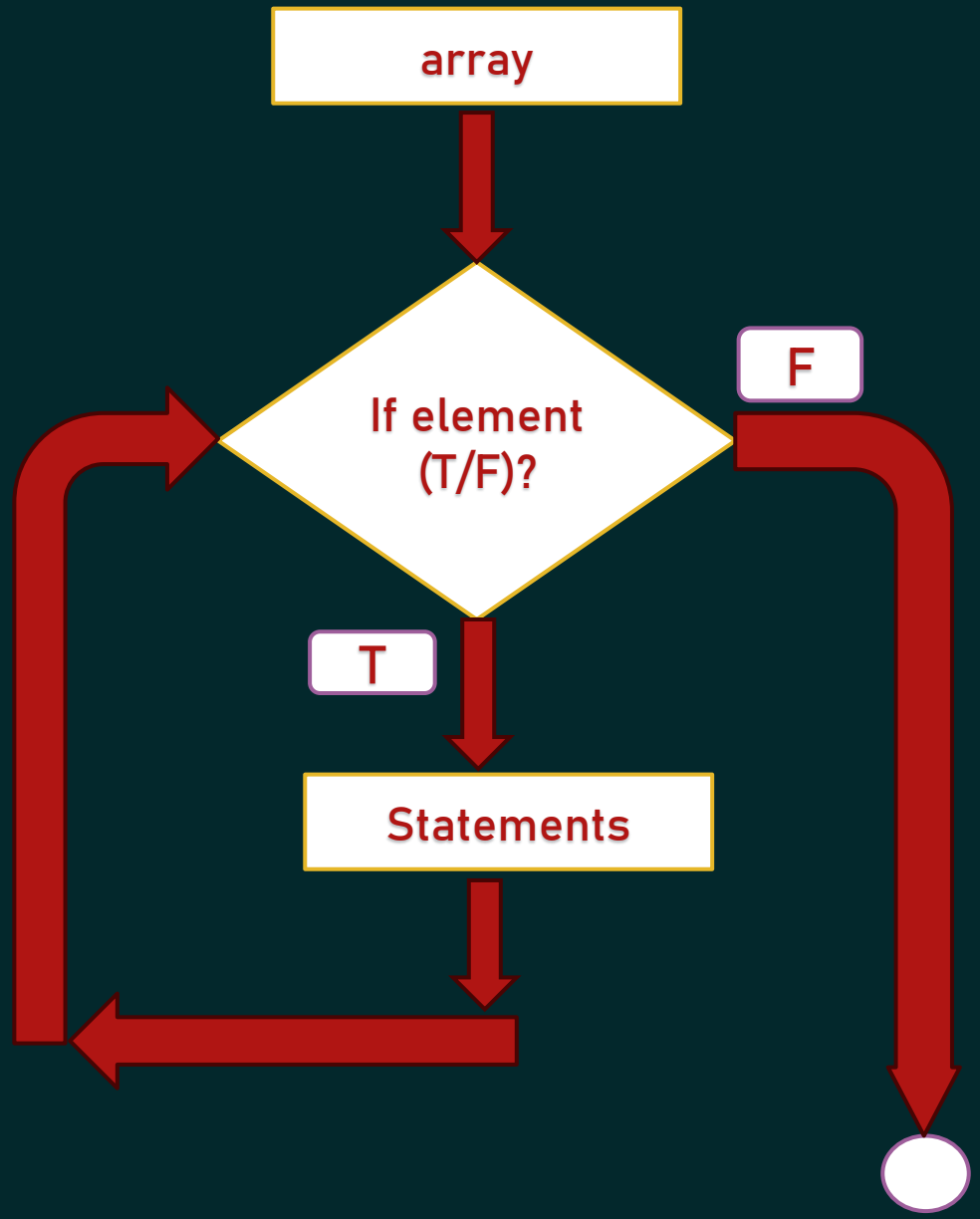
Vinila

for each statement

Syntax:

```
for (type var : array)
{
    statements using var;
}
```

- Also called enhanced for loop



Example-8

- Problem Statement : finding sum of an array of elements.

```
class ForEachExample1{
  public static void main(String args[]){
   int arr[]={12,13,14,44};
   int total=0;
   for(int i:arr){
     total=total+i;
   }
  System.out.println("Total: "+total);
 }
}
```

Output
Total: 83

Nesting of iterative statement

- All Iterative statements can be nested.

Syntax:

```
while(condition1)
{       while(condition2)
        {
        statement1;
        }
statement2;
}
```

Syntax:

```
do
{       do
        {
        statement1;
        }while(condition1);
}while(condition2);
statement2;
```

Nesting of iterative statement

- All Iterative statements can be nested.

Syntax:

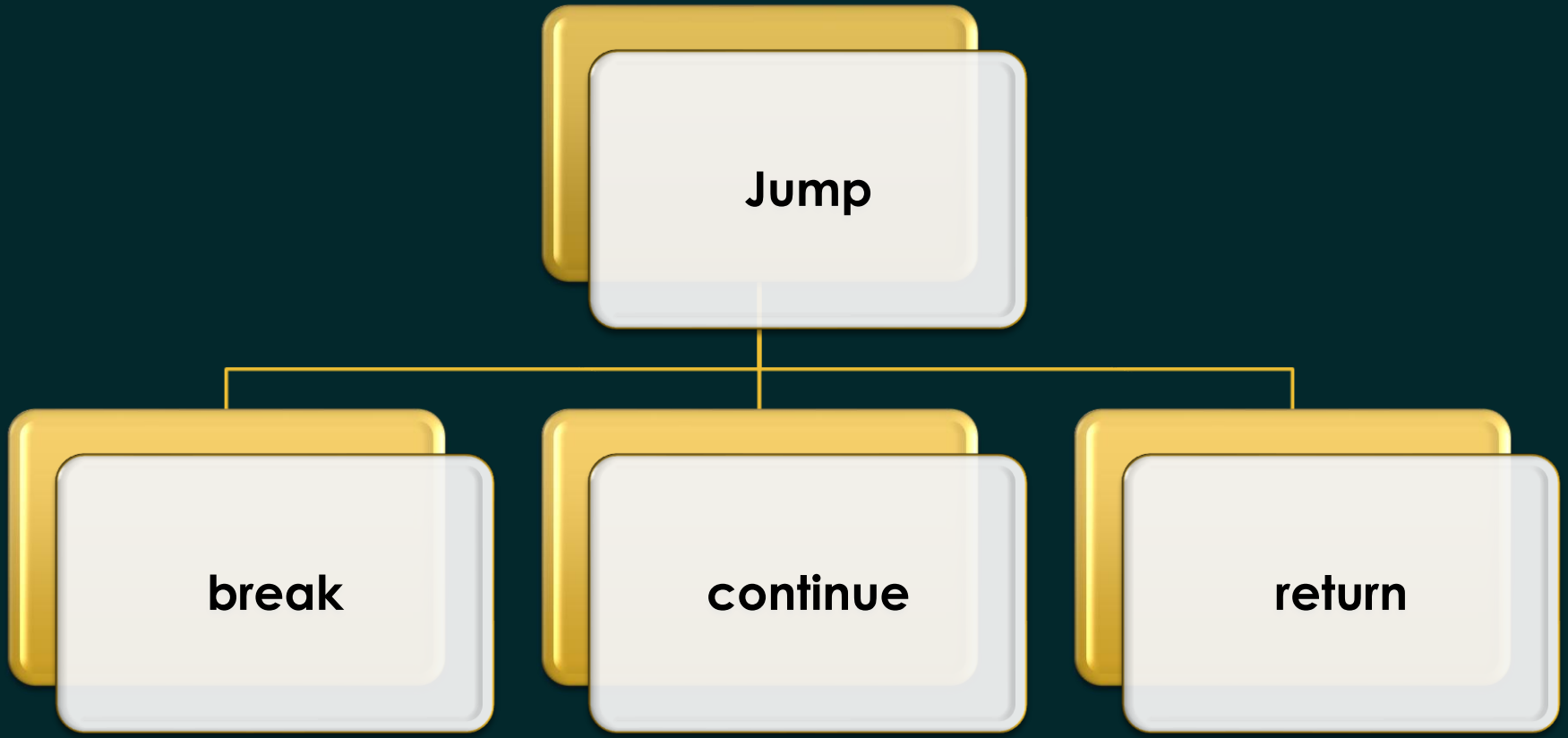
```
for( initialization ; condition; increment/decrement)
{
        for( initialization ; condition; increment/decrement)
        { statement1; }
        statement2;
}
statement3;
```

Syntax:

```
for (type var1 : array1)
{
        for (type var2 : array2)
        {
         statements using var2;
        }
   statements using var1;
}
```

Jump Statement

- Transfer control to some other part of the program



break statement

3 uses of Break Statements

- Exiting a Loop
- As a form of Goto
- In a switch case

Syntax:

```
break;
break label;
```

Example-9

Problem Statement : using break to exit an infinite loop.

```
class break_statement
{
  public static void main(String[] args)
{
    int i=1;
    for (;;i++)
    {
    if (i==5)
     {
     break;
     }
     else
     {
     System.out.println(i);
     } } } }
```

Example-10

Problem Statement : Use Break as a form of goto

```
class break_statement_goto
{
public static void main(String[] args)
{
      Vit_label:
          {
           for(int i=0; i<100; i++)
            {
             System.out.println("Inside Label_3");
              if (i==3)
              break Vit_label;
            }
           }
}
}
```

Example-11

Problem Statement : Use Break in a Switch case

```
class Switch_Case
{
  public static void main(String[] args)
  {
    char a = 'C';
    switch (a)
    {
      case 'A':
        System.out.println("Letter A");
        break;       //break statement to come out of switch
      case 'C':
        System.out.println("Letter C");
        break;
       default:
        System.out.println("Default case: NO Letter Matched!");
    } } }
```

continue statement

- Stops iterations in looping statements.
- Stops iterations of multiple loop using labelled continue

Syntax:

```
continue;
continue label;
```

Example-12

Problem Statement : Use Continue in looping

```
class continue_statement
{
 public static void main(String[] args)
  {
   for(int i=0; i<=10;i++)
    {
     if (i<3)
      {
       Continue; //continues and goes to next iteration
      }
     System.out.println(i);
    }
  }
}
```

Output

```
3
4
5
7
8
9
10
```

Example-13

Problem Statement : Use labelled Continue in looping

```
class continue_label
{
 public static void main(String[] args)
  {
   outer:
       for(int i=0; i<10;i++){
             for(int j=0; j<10;j++) {
                   if (j>i){
                          System.out.println();
                         continue outer;
                         }
              System.out.print( " " +  i*j );
     } }
     System.out.println();
    }   }
```

Output

```
0
0 1
0 2 4
0 3 6 9
0 4 8 12 16
0 5 10 15 20 25
0 6 12 18 24 30 36
0 7 14 21 28 35 42 49
0 8 16 24 32 40 48 56 64
0 9 18 27 36 45 54 63 72 81
```

return statement

- Helps to to return from a method to the caller.

Syntax:

```
return;
```

```
class return_statement
{
 public static void main(String[] args)
  {
   int age = 12;
   System.out.println("Using Return");
   if (age<18)
   return; //terminates the method
   System.out.println("Will not get executed!");
  }
}
```

Summary

- Iterative statements
- Jump Statements
- Example programs

Arrays in JAVA

- What is Array?
- Types of Array.
- Working with Array.
- Arrays class in Java
- Example Programs.

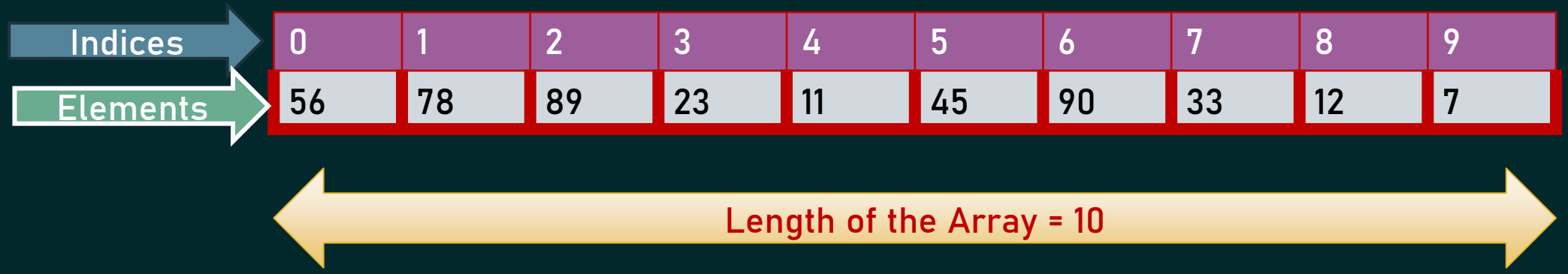
Array

- Data structure used to store and manipulate collections of elements of same type in a contiguous memory location.

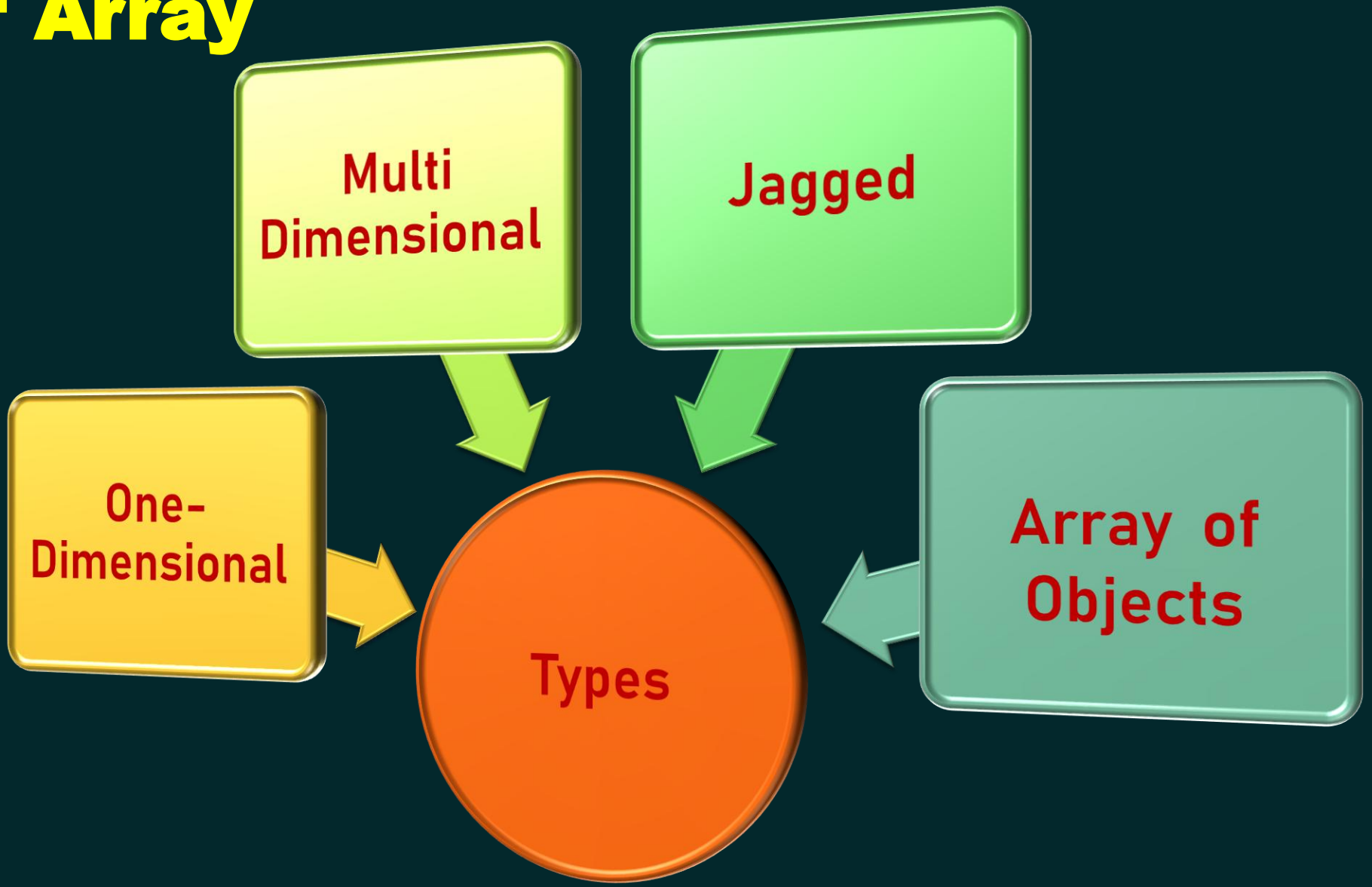


Array

- Array in Java is index-based, the first element of the array is stored at the 0th index, 2nd element is stored on 1st index and so on.
- Array Representation :



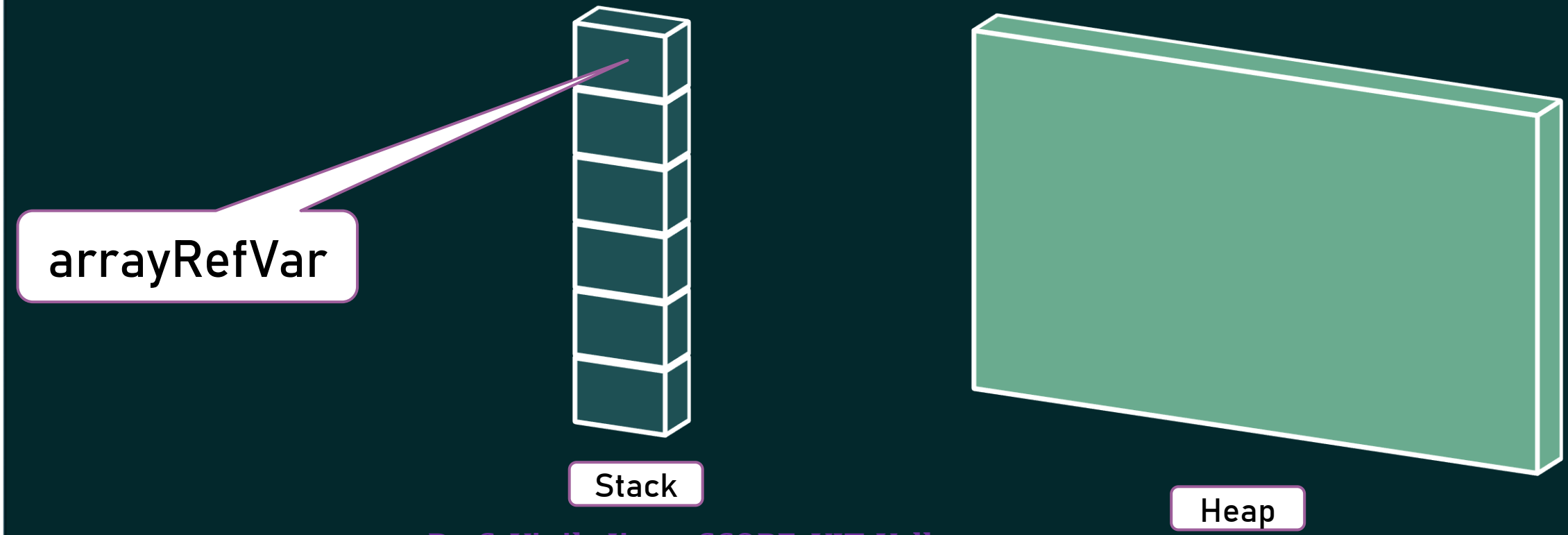
Types of Array



Single Dimensional Array

Syntax:

```
dataType[] arrayRefVar; // preferred way
dataType arrayRefVar[]; // works but not preferred way
```



Creating an Array

Using new operator.

Syntax:

arrayRefVar = new dataType[arraySize];

dataType[] arrayRefVar = new dataType[arraySize];

dataType[] arrayRefVar = {value0, value1, ..., valuek};

Eg. double[] myList = new double[10];

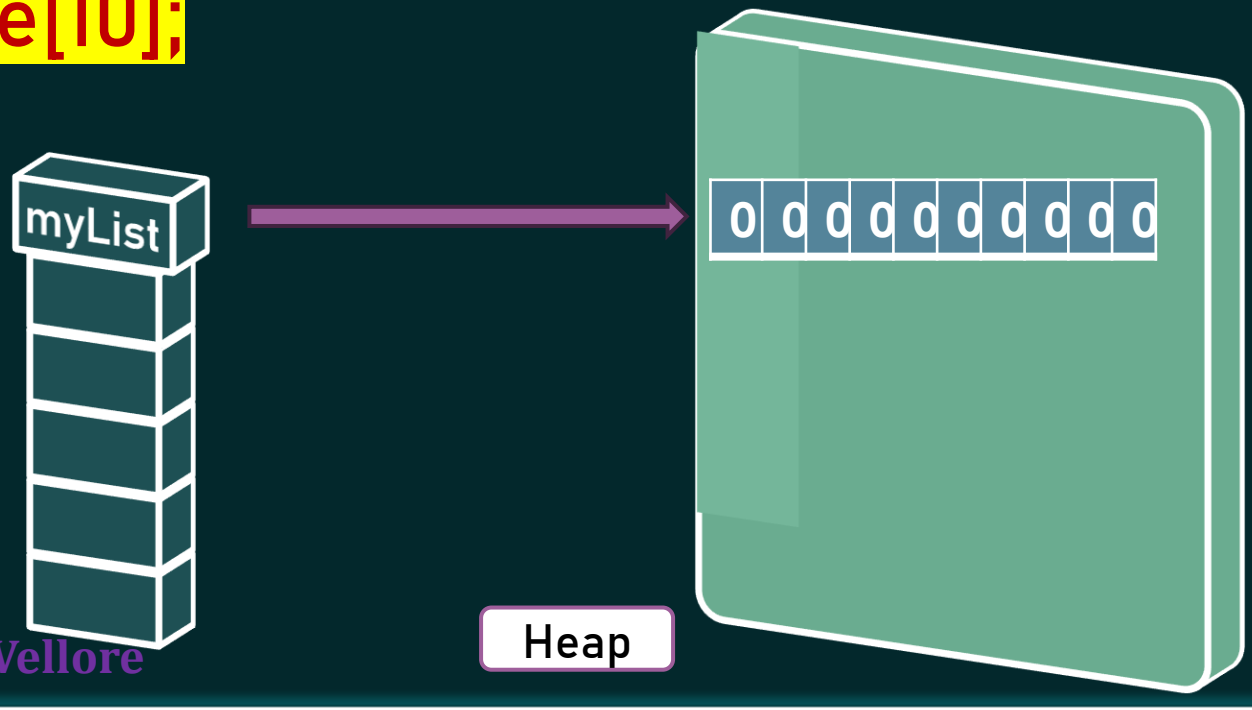
Creating an Array

Using new operator.

Syntax:

```
arrayRefVar = new dataType[arraySize];
dataType[] arrayRefVar = new dataType[arraySize];
```

Eg. `double[] myList = new double[10];`



Initializing an Array

Syntax:

dataType[] arrayRefVar = {value1, value2, value3....}

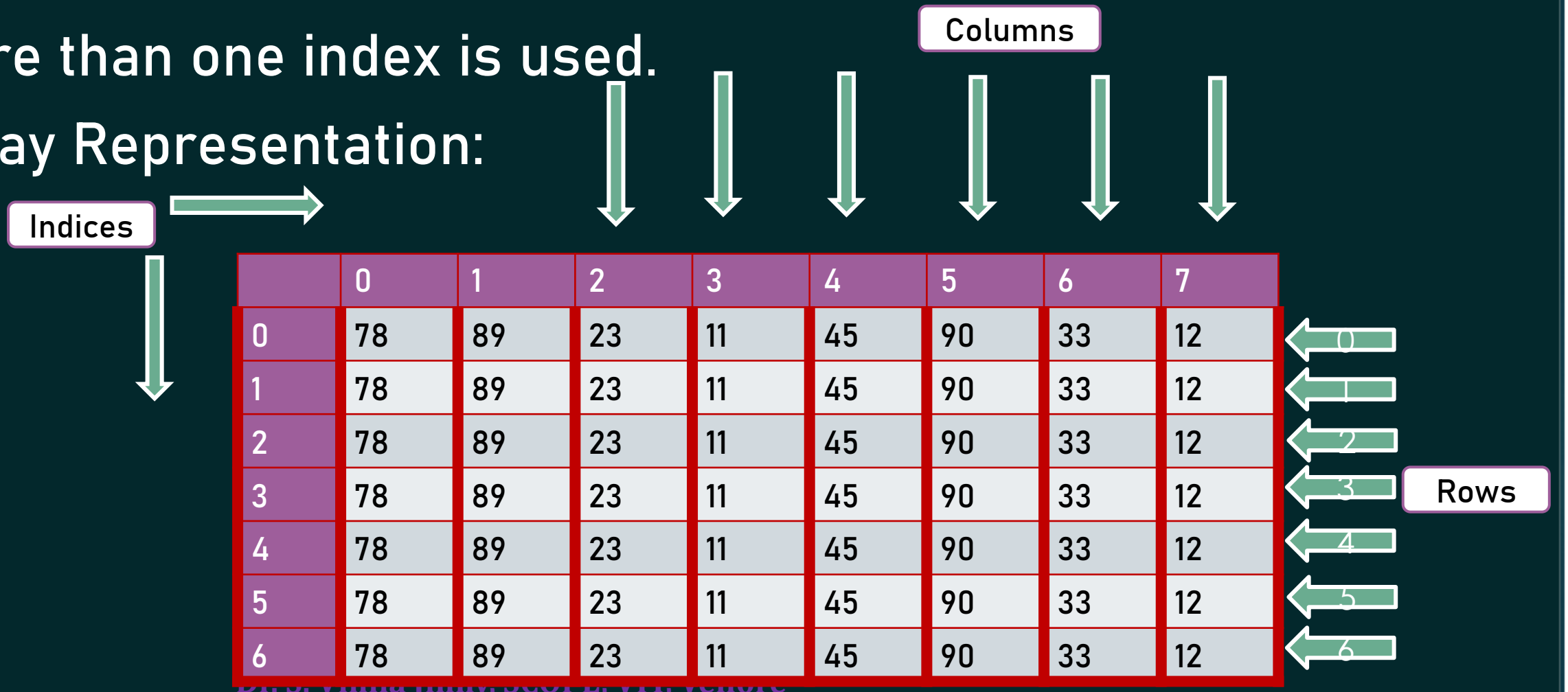
Eg. double[] myList = {34.5, 67.8, 12.5, 5.9, 3.2, 2.8, 9.4, 12.8}

```
public class ArrayAverage {
    public static void main(String[] args) {
        int[] array = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 };
        int sum = 0;
        int length=array.length;
        for (int i = 0; i < 12; i++) {
          sum += array[i];
         }
       double average = sum / length;
       System.out.println("Average of array : "+average);
    }
}
```

Output:
Average of array : 6.0

Multi-dimensional Array

- Implemented as Arrays of Arrays.
- More than one index is used.
- Array Representation:



	0	1	2	3	4	5	6	7
0	78	89	23	11	45	90	33	12
1	78	89	23	11	45	90	33	12
2	78	89	23	11	45	90	33	12
3	78	89	23	11	45	90	33	12
4	78	89	23	11	45	90	33	12
5	78	89	23	11	45	90	33	12
6	78	89	23	11	45	90	33	12

Multi-dimensional Array

- Syntax:
  - datatype refVaraiable = new datatype[size][size];
- Example:
- Method1:
  - int twoD [][] = new int [4][5];
- Method2:
  - int twoD [][] = new int [2][];
  - twoD[0]= new int [5];
  - twoD[1]= new int [5];

Multi-dimensional Array – Example Program

```
public class ChessboardPattern {
    public static void main(String[] args) {
        char[][] chessboard = new char[8][8];
         for (int row = 0; row < 8; row++) {
           for (int col = 0; col < 8; col++) {
              if ((row + col) % 2 == 0) {
                 chessboard[row][col] = 'W';
              } else {
                 chessboard[row][col] = 'B';
              }
              System.out.print(chessboard[row][col] + " ");
           }
           System.out.println();
        }
    }
}
```

Output

```
W B W B W B W B
 B W B W B W B W
 W B W B W B W B
B W B W B W B W
W B W B W B W B
 B W B W B W B W
W B W B W B W B
B W B W B W B W
```

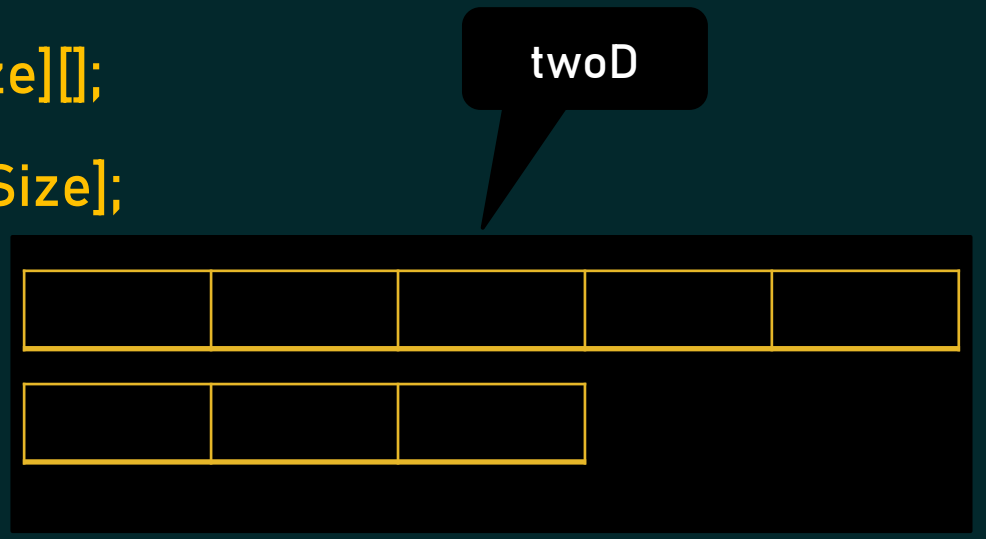
Jagged Array

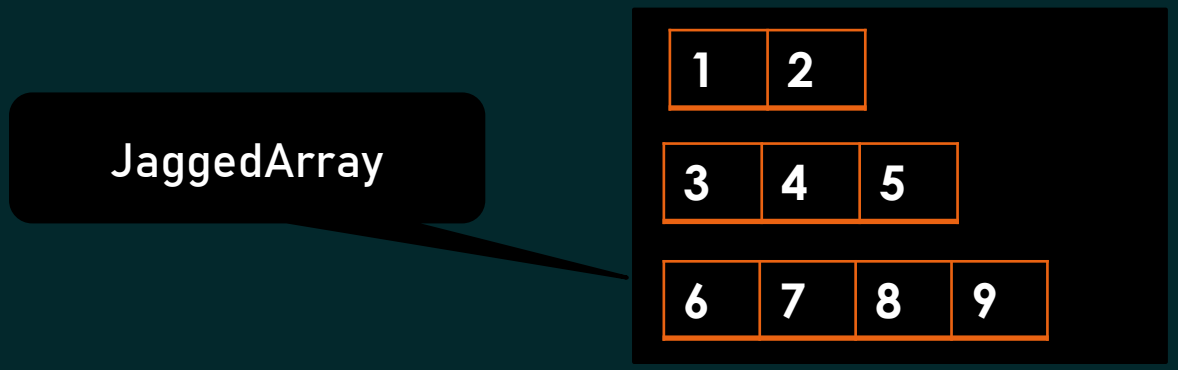
- A jagged Array is an array of arrays where each element is an array.
- It is a special type of Multidimensional array with variable number of columns in each row.
- Jagged arrays are also known as "ragged arrays" or "irregular arrays".

	Column-1	Column-2	Column-3	Column-4	Column-5
Row-1	45	67	78		
Row-2	45	67	89	12	09
Row-3	33	55			
Row-4	23	61			

Jagged Array

- Syntax:
  - dataType[][] jaggedArray = new dataType[rowSize][];
  - jaggedArray[rowIndex] = new dataType[columnSize];
- Example:
- Method1:
  - int twoD [][] = new int [2][];
  - twoD[0]= new int [5];
  - twoD[1]= new int [3];
- Method2:
- int[][] jaggedArray = { {1, 2}, {3, 4, 5}, {6, 7, 8, 9} };





Jagged Array – Example Program

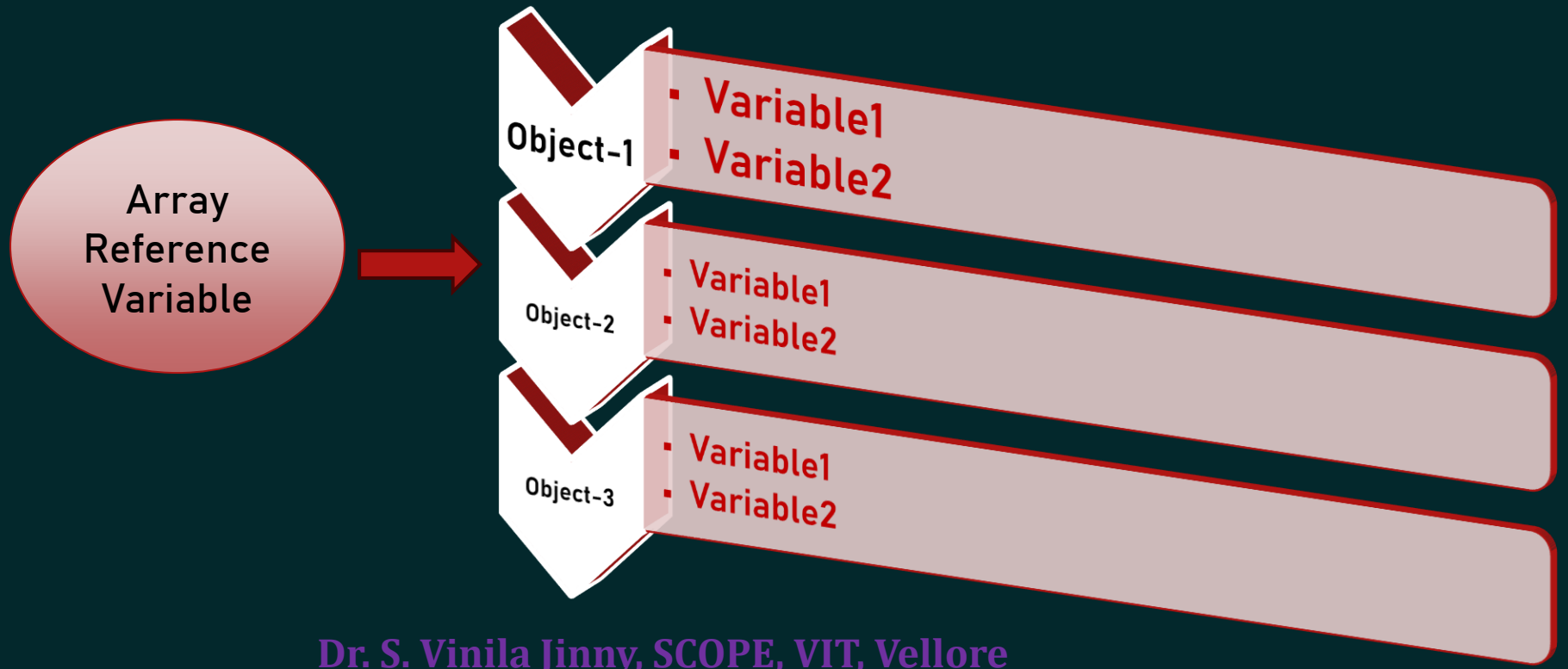
```
class Main {
    public static void main(String[] args) {
        int r = 5;
        int arr[][] = new int[r][];
        for (int i = 0; i < arr.length; i++)
            arr[i] = new int[i + 1];
        int count = 0;
        for (int i = 0; i < arr.length; i++)
            for (int j = 0; j < arr[i].length; j++)
                arr[i][j] = count++;
        System.out.println("Contents of 2D Jagged Array");
        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr[i].length; j++)
                System.out.print(arr[i][j] + " ");
            System.out.println();            }     }     }
```

Output

```
Contents of 2D Jagged Array
0
1 2
3 4 5
6 7 8 9
10 11 12 13 14
```

Array of Objects

- An array that contains class type elements are known as an array of objects.
- It stores the reference variable of the object.



Array of Objects

Syntax:
ClassName [ ] object_Name = new ClassName [length of Array];

- Example Program

```
public class ArrayOfObjects {
    public static void main(String args[]) {
        Person[] peopleArray = new Person[3];
        peopleArray[0] = new Person("Alice", 25);
        peopleArray[1] = new Person("Bob", 30);
        peopleArray[2] = new Person("Charlie", 22);
        System.out.println("Person Object 1:");
        peopleArray[0].display();
        System.out.println("Person Object 2:");
        peopleArray[1].display();
        System.out.println("Person Object 3:");
        peopleArray[2].display();   } }
```

Array of Objects

Syntax:
ClassName [ ] object_Name = new ClassName [length of Array];

```
class Person {
    String name;
    int age;
    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    public void display() {
        System.out.print("Name = " + name + "  " + " Age = " + age);
        System.out.println();
    }
}
```

Output:
Person Object 1:
Name = Alice Age = 25

Person Object 2:

Name = Bob Age = 30

Person Object 3:

Name = Charlie Age = 22

Arrays class in Java

- The Arrays class in Java is a utility class in the java.util package that provides various static methods to manipulate arrays (such as sorting, searching, comparing, and filling arrays).
- It works with arrays of primitive data types (like int[], double[], etc.) and object arrays (like String[], Integer[], etc.).

```
import java.util.Arrays;
```

Arrays class in Java

Method	Description
sort()	Sorts the array in ascending order.
binarySearch()	Searches for a value in a sorted array using binary search.
equals()	Checks if two arrays are equal.
fill()	Fills all elements of an array with a specified value.
copyOf()	Copies an array to a new array with a specified length.
copyOfRange()	Copies a specified range from an array.
toString()	Returns a string representation of the array.
asList()	Converts an array to a List (for object arrays only).
compare()	Compares two arrays lexicographically. (Java 9+)
mismatch()	Finds the index of the first mismatch between two arrays. (Java 9+)

Arrays class in Java

```
import java.util.Arrays;
public class ArraysClassExample {
    public static void main(String[] args) {
        int[] numbers = {5, 2, 9, 1, 5, 6};
        // Sorting the array
        Arrays.sort(numbers);
System.out.println("Sorted array: " + Arrays.toString(numbers));
        // Binary Search (must be sorted)
        int index = Arrays.binarySearch(numbers, 5);
        System.out.println("Index of 5: " + index);
       // Fill array with a value
        int[] filledArray = new int[5];
        Arrays.fill(filledArray, 7);
       System.out.println("Filled array: " +Arrays.toString(filledArray));
// Copy array
 int[] copy = Arrays.copyOf(numbers, numbers.length);
 System.out.println("Copied array: " + Arrays.toString(copy));

        // Check equality
System.out.println("Arrays equal? " +Arrays.equals(numbers, copy));

        // Convert to List
        String[] names = {"Alice", "Bob", "Charlie"};
  System.out.println("Array as List: " + Arrays.asList(names));
    }
}
```

Summary

- What is Array?
- Types of Array.
- Working with Array.
- Example Programs.

JAVA STRING

- What is a java string?
- Creation of String
- Classes Supporting String
- Methods Supporting String
- Example Program Using String

What is a Java String?

- A String is a sequence of characters. Char type represents only one character.
- Easiest way to represent sequence of character is char array

```
char sample[]=new char[10];
```

- but it doesn't support string operations.
- Strings in Java are represented using a String Class, which is a predefined class in Java Library.

Java String Supporting Classes

- 3 classes support String Creation
  - Class String
  - Class StringBuffer
  - Class StringBuilder

All belongs to java.lang package

String Class

- String is not a Primitive type but it is a reference type.
- The variable declared by a reference type is known as a reference variable that references an object.
- String objects are immutable, which means they can't be overwritten.
- Every time we try to alter the contents of a String object, a new object is created.
- Each character in String is a 16-bit Unicode Character.

String using String class

- String Literal

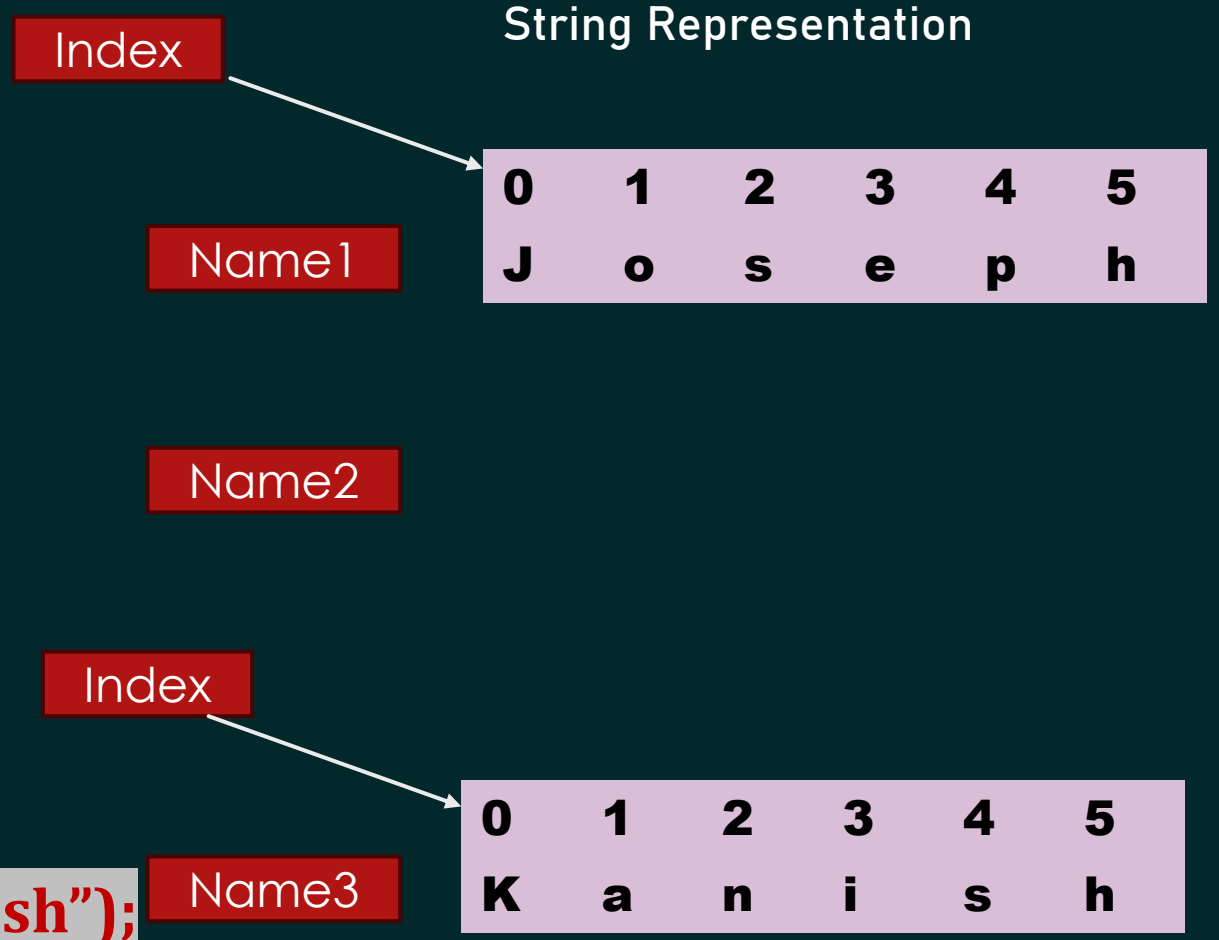
Example:

```
String Name1="Joseph";
```

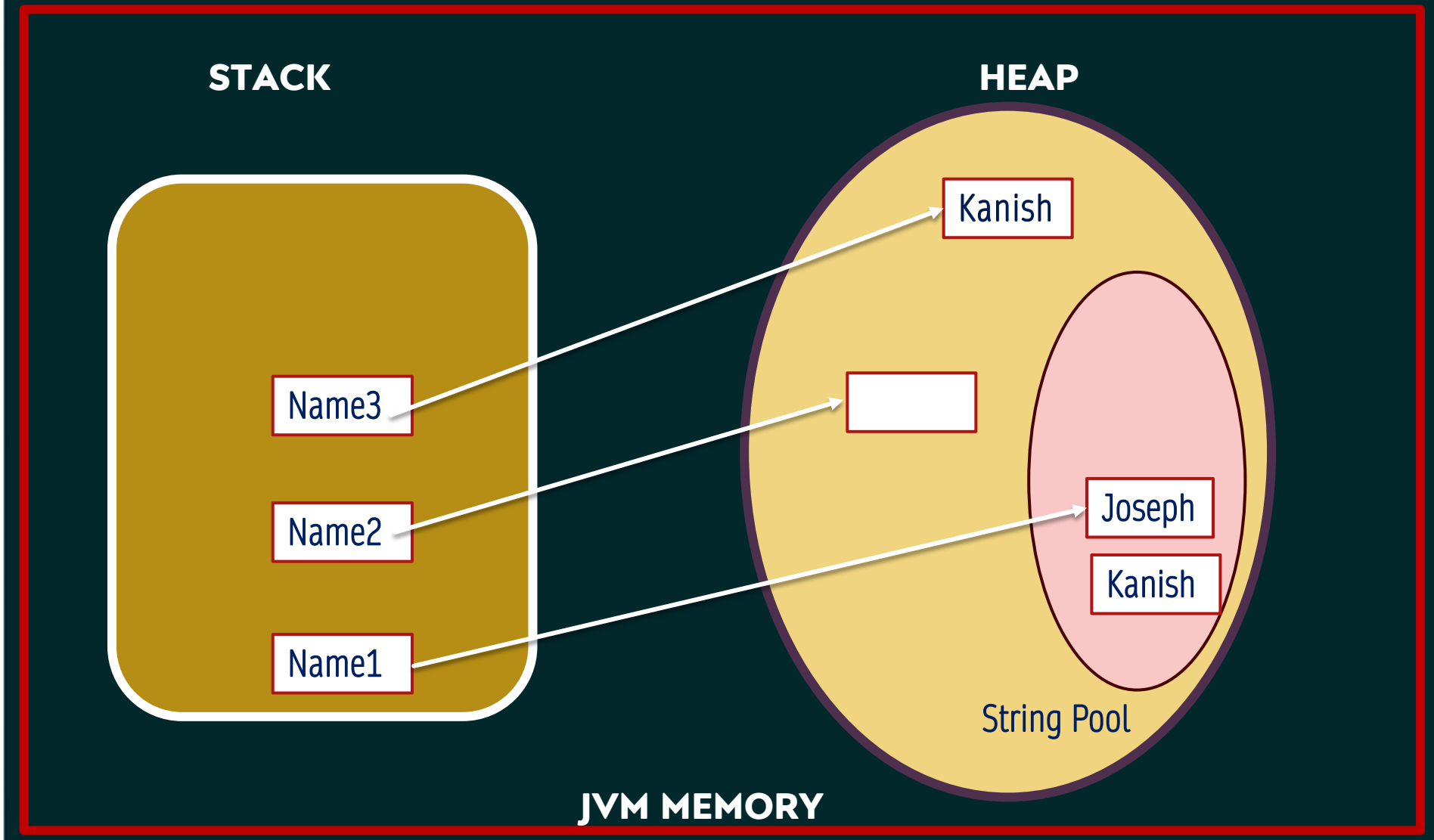
- Using new Keyword

Example:

```
String Name2=new String();
String Name3=new String("Kanish");
```



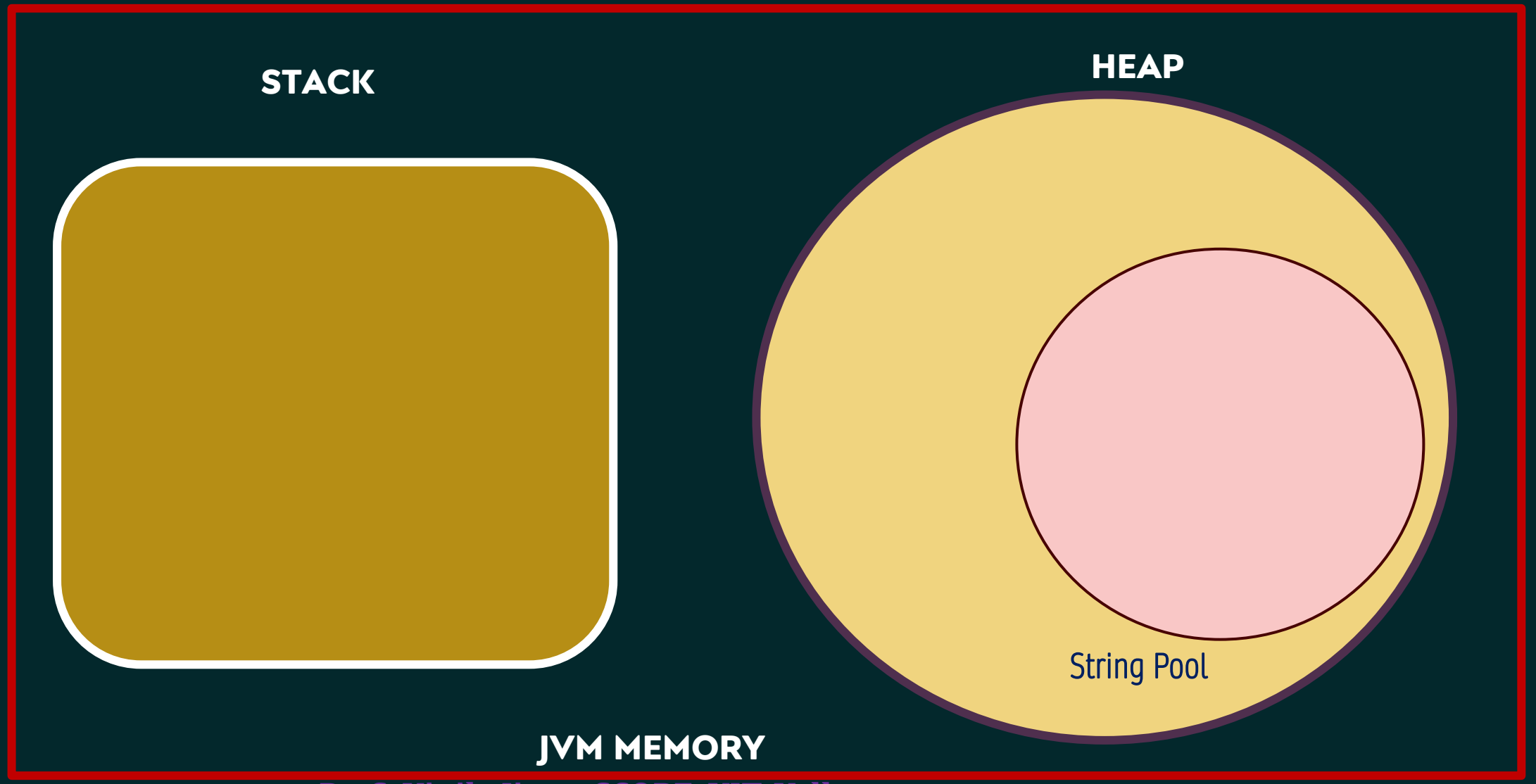
Storage of Strings - Example




```
String Name1="Joseph";
String Name2=new String();
String Name3=new String("Kanish");
```

Total number of objects in memory
4

Storage of Strings



JVM memory

- STACK MEMORY
  - Stack Memory is used for execution of the program.
  - All primitive variables find its place in stack.
- HEAP
  - Heap memory is used to store the data of program.
  - All non-primitive variables that is objects finds its place in the heap which is a large memory.
  - A reference to this objects is maintained in Stack Memory.

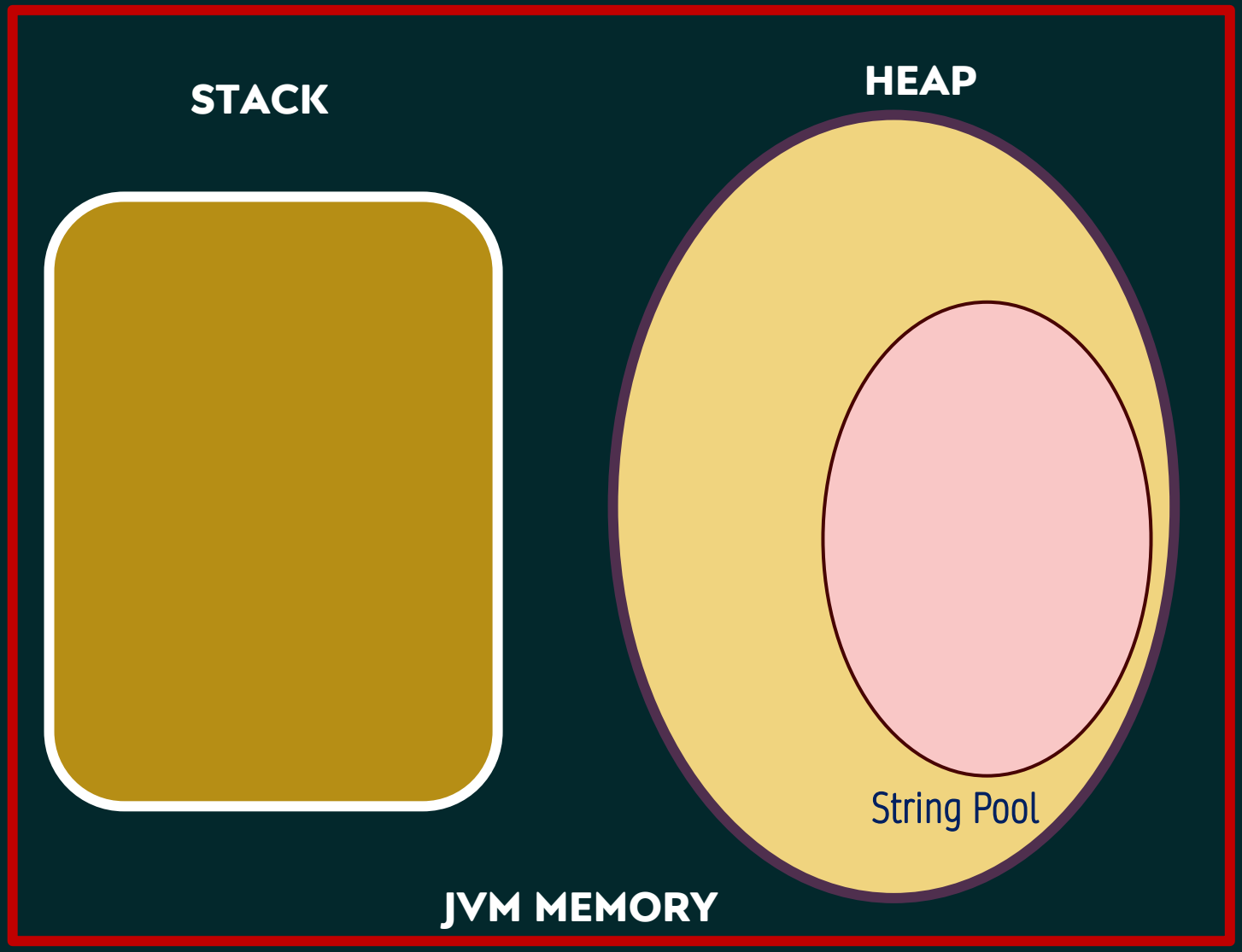
String pool

String Pool will be initially empty.

- Strings are everywhere in Java, they use up a lot of memory.
- The string pool, also known as the intern pool, is a location in the heap that collects all these strings.
- When String is created as a Literal, JVM looks in the String pool to find if any other String is stored with same value.
- If found, it just returns the reference to that String object else it creates a new String object with given value and stores it in the String pool.

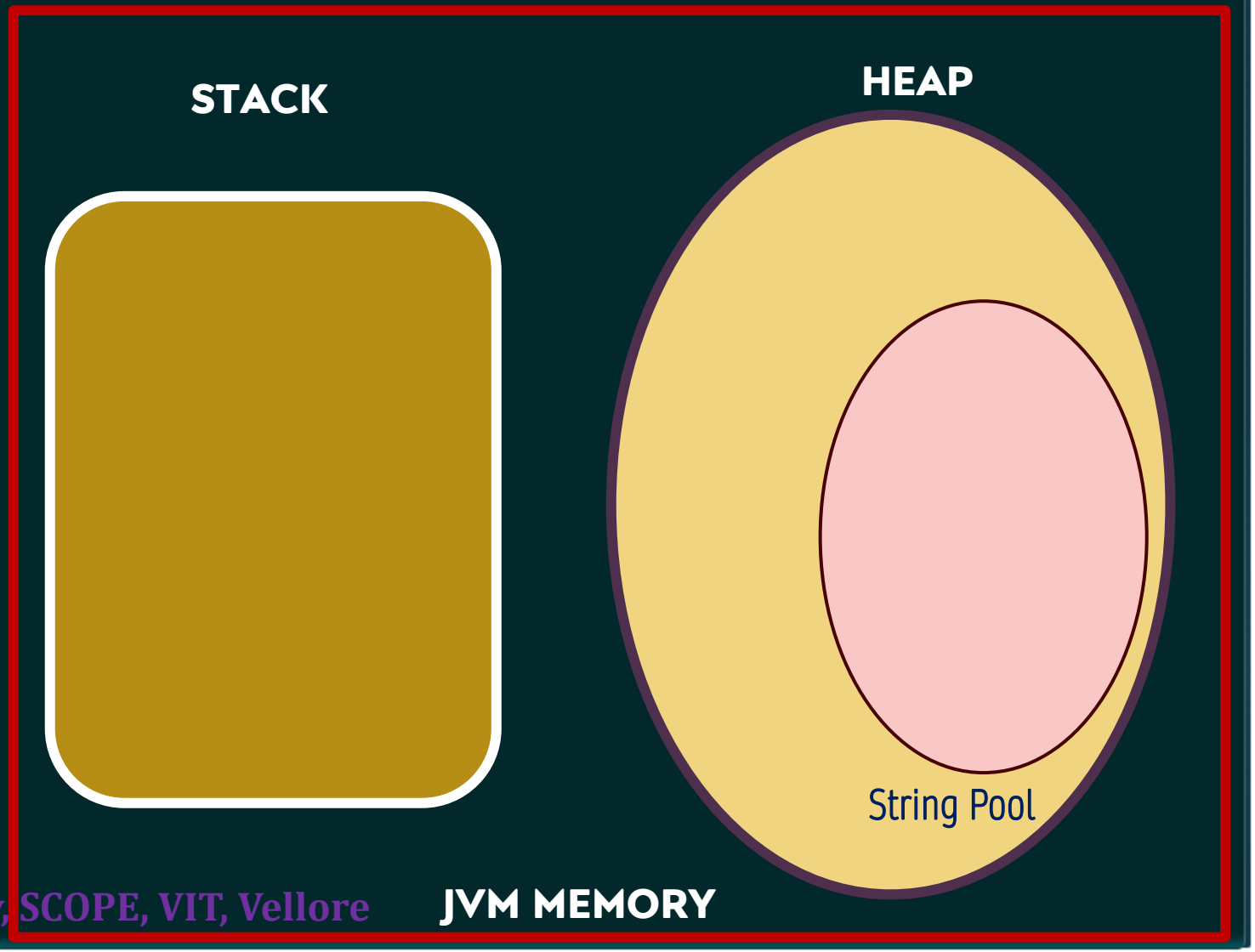
Example

- String s1=new String(“VIT”);
- String s2=“University”;
- String s3=“University”;
- String s4=new String(“Vellore”);
- String s5=“VIT”;
- String s6=s5;
- String s2=“hello”;



Immutability

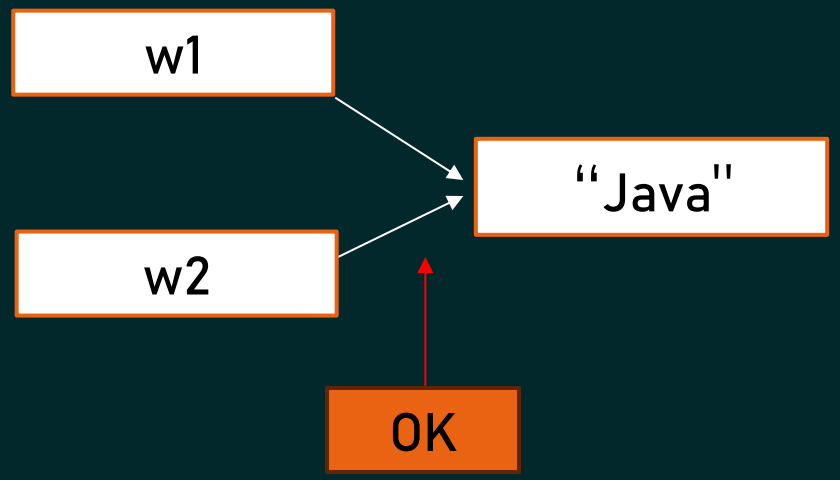
- Strings are immutable. Immutable objects are ones that cannot be modified after they have been created
- String x = "Java";
- x.concat(" Rules!");
- System.out.println("x = " + x);



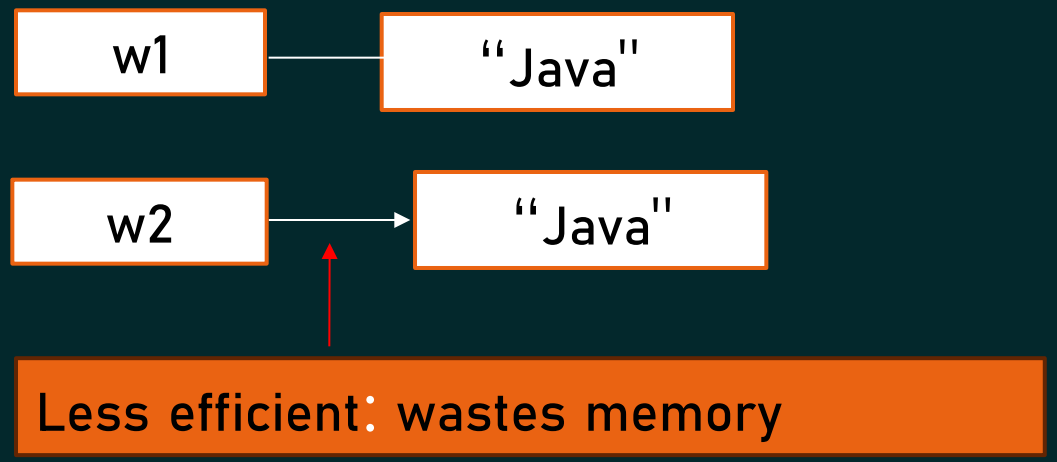
Advantages Of Immutability

Uses less memory.

```
String w1 = "Java";
String w2 = w1;
```



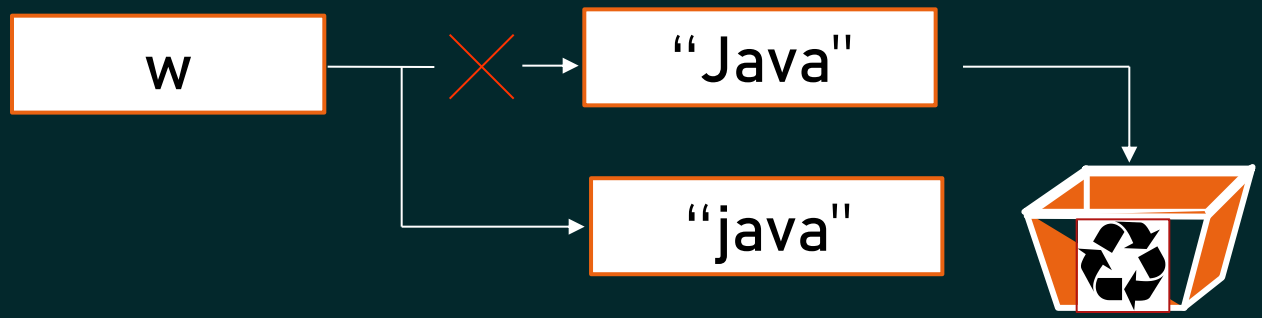

```
String w1 = "Java";
String w2 = new String(w1);
```



Disadvantages of Immutability

Less efficient — you need to create a new string and throw away the old one even for small changes.

```
String w = “Java";
w =  w.toLowercase();
```



String Constructors

- Class String Provides many constructors

Commonly used Constructors :

- String()
- String(String str)
- String(char chars[ ])
- String(char chars[ ], int startIndex, int count)
- String(byte byteArr[ ])
- String(byte byteArr[ ], int startIndex, int count)

String Constructors

```
 public class StringConstructors {
 public static void main( String args[] )
{
 char charArray[] = { 'b', 'i', 'r', 't', 'h', ' ', 'd', 'a', 'y' };
 byte byteArray[] = { ( byte ) 'n', ( byte ) 'e', ( byte ) 'w', ( byte ) ' ', ( byte ) 'y',  ( byte ) 'e', ( byte ) 'a', ( byte ) 'r' };
String s = new String( "hello" );
String s1 = new String( );
String s2 = new String( s );
String s3 = new String( charArray );
String s4 = new String( charArray, 6, 3 );
String s5 = new String( byteArray, 4, 4 );
String s6 = new String( byteArray );
String output = "s1 = " + s1 + "\ns2 = " + s2 + "\ns3 = " + s3 + "\ns4 = " + s4 + "\ns5 = " + s5 + "\ns6 = " + s6;
}
}
```

Methods in String Class

- String Length
  - int length( )

str → Vit University

```
System.out.print(str.length());
```

14

```
System.out.print( “Vit”.length());
```

3

- String Concatenation
  - + operator, which concatenates two strings

```
String number=“2”;
System.out.print( “There are” + number + “flowers”);
```

There are 2 flowers

```
int number= 2;
System.out.print( “There are” + number + number +  “flowers”);
```

There are 22 flowers

Methods in String Class

- String Conversion
  - String toString()

```
class Student{
 int rollno;
 String name;
 String city; Student(int rollno, String name, String city){
 this.rollno=rollno;
 this.name=name;
 this.city=city;
 }
  public String toString(){
  return rollno+" "+name+" "+city;
 }
 public static void main(String args[]){
   Student s1=new Student(111,“Nidhi",“Bangalore");
   Student s2=new Student(222,“Dhanya",“Delhi");
   System.out.println(s1);
   System.out.println(s2);
 }
}
```

Output
111 Nidhi Bangalore
222 Dhanya Delhi

Methods in String Class

- Character Extraction
  - charAt(int where)

```
String myStr = "Hello";
char result = myStr.charAt(0);
System.out.println(result);
```

H

  - void getChars(int sourceStart, int sourceEnd, char target[ ], int targetStart)

```
char[] myArray = {'0', '1', '2', '3', '4', '5', '6', '7', '8', '9'};
System.out.println(myArray);
String myStr = "Hello, World!";
myStr.getChars(7, 12, myArray, 4);
System.out.println(myArray);
```

0123456789
0123World9

Methods in String Class

- Character Extraction
  - byte[ ] getBytes( )

```
String myStr = "Hello";
byte[] result = myStr.getBytes();
System.out.println(result[0]);
```

72

  - char[ ] toCharArray( )

```
String myStr = "Hello";
char[] myArray = myStr.toCharArray();
System.out.println(myArray[0]);
```

H

Methods in String Class

- String Comparison
  - boolean equals(Object str)

```
String myStr1 = "Hello";
String myStr2 = "Hello";
String myStr3 = "Another String";
System.out.println(myStr1.equals(myStr2));
System.out.println(myStr1.equals(myStr3));
```

true
false

  - boolean equalsIgnoreCase(String str)

```
String myStr1 = "Hello";
String myStr2 = "HELLO";
String myStr3 = "Another String";
System.out.println(myStr1.equalsIgnoreCase(myStr2));
System.out.println(myStr1.equalsIgnoreCase(myStr3));
```

true
false

Methods in String Class

- String Comparison
  - boolean regionMatches(int startIndex, String str2,int str2StartIndex, int numChars)

```
String myStr1 = "Hello, World!";
String myStr2 = "New World";
System.out.println(myStr1.regionMatches(7, myStr2, 4, 5));
System.out.println(myStr1.regionMatches(0, myStr2, 0, 5));
```

```
true
false
```

  - boolean regionMatches(boolean ignoreCase,int startIndex, String str2,int str2StartIndex, int numChars)
  - boolean startsWith(String str)
  - boolean endsWith(String str)
  - int compareTo(String str)

```
String myStr = "Hello";
System.out.println(myStr.startsWith("Hel"));
System.out.println(myStr.startsWith("llo"));
System.out.println(myStr.endsWith("Hel"));
System.out.println(myStr.endsWith("llo"));
```

```
true
false
False
true
```

Methods in String Class – String Comparison

- 3 ways to compare strings
  - By equals() method (inherited from Object)
  - By == operator
  - By CompareTo() method

Methods in String Class – String Comparison

```
class Teststringcomparison1{
 public static void main(String args[]){
   String s1="Jinu";
   String s2="Jinu";
   String s3=new String("Jinu");
   String s4="Minu";
   String s5="MINU";
   System.out.println(s1.equals(s2));
   System.out.println(s1.equals(s3));
   System.out.println(s1.equals(s4));
   System.out.println(s4.equals(s5));
   System.out.println(s1.equalsIgnoreCase(s2));
 }
}
```

- By equals() method
- public boolean equals(Object another)
- public boolean equalsIgnoreCase(String another))

Output

true
true
false
false
true

Methods in String Class – String Comparison

- By == operator

```
class Teststringcomparison3{
 public static void main(String args[]){
   String s1="Jinu";
   String s2="Jinu";
   String s3=new String("Jinu");
   String s4="Minu";
   System.out.println(s1==s2);
   System.out.println(s1==s3);
   System.out.println(s1==s4);
 }
}
```

Output

true
false
false

Methods in String Class – String Comparison

- By using CompareTo method

```
class Teststringcomparison4{
 public static void main(String args[]){
   String s1="Jinu";
   String s2="Jinu";
   String s3="Minu";
   System.out.println(s1.compareTo(s2));
   System.out.println(s1.compareTo(s3));
   System.out.println(s3.compareTo(s1));
 }
}
```

Output

0

-1

1

Methods in String Class

- String Searching
  - int indexOf(int *ch*)
  - int lastIndexOf(int *ch*)
  - int indexOf(String *str*)
  - int lastIndexOf(String *str*)
  - int indexOf(int *ch*, int *startIndex*)
  - int lastIndexOf(int *ch*, int *startIndex*)
  - int indexOf(String *str*, int *startIndex*)
  - int lastIndexOf(String *str*, int *startIndex*)

```
String myStr = “Welcome Vitian. Vit a place to learn";
System.out.println(myStr.indexOf(“Vit"));
System.out.println(myStr.lastIndexOf(“Vit"));
```

```
8
16
```

Methods in String Class

- Modifying a String
  - String substring(int *startIndex*)
  - String substring(int *startIndex*, int *endIndex*)

```
String myStr = "Hello, World!";
System.out.println(myStr.substring(7, 12));
```

World

  - String concat(String *str*)

```
String firstName = “Vit";
String lastName = “Scope";
System.out.println(firstName.concat(lastName));
```

Vit Scope

Methods in String Class

- Modifying a String
  - String replace(char *original*, char *replacement*)

```
String myStr = “Vit Scope";
System.out.println(myStr.replace(‘p', ‘r'));
```

Vit Score

  - String trim( )

```
String myStr = "      Hello World!      ";
System.out.println(myStr);
System.out.println(myStr.trim());
```

Hello World
Hello World

Methods in String Class

- Data Conversion
- static String valueOf(double *num*)
- static String valueOf(long *num*)
- static String valueOf(Object *ob*)
- static String valueOf(char *chars*[ ])

```
char[] myArray = {'a', 'b', 'c'};
System.out.println(String.valueOf(myArray));
System.out.println(String.valueOf('A'));
System.out.println(String.valueOf(true));
System.out.println(String.valueOf(4.5f));
System.out.println(String.valueOf(5.2));
System.out.println(String.valueOf(12));
System.out.println(String.valueOf(1400L));
```

```
abc
A
true
4.5
5.2
12
1400
```

Methods in String Class

- Changing Case
  - String toLowerCase( )
  - String toUpperCase( )

```
String txt = "Hello World";
System.out.println(txt.toUpperCase());
System.out.println(txt.toLowerCase());
```

HELLO WORLD
hello world

- Joining String
  - static String join(CharSequence delim, CharSequence … strs)

```
String fruits = String.join(" ", "Orange", "Apple", "Mango");
System.out.println(fruits);
```

Orange Apple Mango

Methods in String Class

- Additional Methods
  - public boolean contains(CharSequence chars)

```
String myStr = "Hello";
System.out.println(myStr.contains("Hel"));   // true
System.out.println(myStr.contains("e"));     // true
System.out.println(myStr.contains("Hi"));    // false
```

true
true
false

  - public String[] split(String regex, int limit)

```
String myStr = "Split a string by spaces, and also punctuation.";
String regex = "[,\\.\\s]";
String[] myArray = myStr.split(regex);
for (String s : myArray) {
  System.out.println(s);
}
```

Split
a
string
by
spaces

and
also
punctuation

Summary

- What is a java string?
- Creation of String
- Classes Supporting String
- Methods Supporting String
- Example Program Snippets Using String

JAVA STRINGBUFFER & STRING BUILDER

- What is a java Stringbuffer class?
- Constructors and Methods of StringBuffer
- What is a java Stringbuilder class?
- StringBuffer Vs StringBuilder
- Example program Snippets.

String Buffer

- Object that represents a sequence of char values.
- StringBuffer is a mutable Object.
- Synchronized and thus thread safe.

Public final class StringBuffer extends Object implements Serializable, CharSequence

String Buffer Constructors

- 4 constructors
  - StringBuffer( )

```
StringBuffer ob = new StringBuffer();
```

Capacity 16

  - StringBuffer(int capacity)

```
StringBuffer m = new StringBuffer(12);
```

Capacity 12

  - StringBuffer(String str)

```
StringBuffer ob = new StringBuffer(String);
```

  - StringBuffer(CharSequence chars)

```
StringBuffer ob = new StringBuffer(CharSequence);
```

Methods of class String Buffer

- int length( )

```
StringBuffer university=new StringBuffer("VIT");
System.out.println(university.length());
```

3

- int capacity( )

```
StringBuffer university=new StringBuffer("VIT");
System.out.println(university.capacity());
```

19

Methods of class String Buffer

- void ensureCapacity(int capacity)

```
StringBuffer university=new StringBuffer("VIT");
System.out.println(university.capacity());
university.ensureCapacity(50);
System.out.println(university.capacity());
```

```
19
50
```

- void setLength(int len)

```
StringBuffer university=new StringBuffer("VIT");
System.out.println(university.length());
university.setLength(12);
System.out.println(university.length());
university.setLength(1);
System.out.println(university.length());
```

```
3
12
1
```

Methods of class String Buffer

- StringBuffer append(String str)
- StringBuffer append(int num)
- StringBuffer append(Object obj)

```
int a = 42;
StringBuffer sb = new StringBuffer(40);
sb.append("a=").append(a).append("!");
System.out.println(sb);
```

a=42!

Methods of class String Buffer

- StringBuffer insert(int index, String str)
- StringBuffer insert(int index, char ch)
- StringBuffer insert(int index, Object obj)

```
StringBuffer sb = new StringBuffer("I Java!");
System.out.println(sb);
sb.insert(2, "like ");
System.out.println(sb);
```

I like Java!

Methods of class String Buffer

- StringBuffer reverse( )
- StringBuffer delete(int *startIndex*, int *endIndex*)
- StringBuffer deleteCharAt(int *loc*) StringBuffer
- replace(int startIndex, int endIndex, String str)

```
StringBuffer sb = new StringBuffer("Vellore Fort is in Vellore");
System.out.println(sb);  sb.reverse();
System.out.println(sb);  sb.reverse();
System.out.println("Before delete: " + sb);  sb.delete(4, 7);
System.out.println("After delete: " + sb);  sb.deleteCharAt(0);
System.out.println("After deleteCharAt: " + sb);
sb.replace(5, 7, "is");
System.out.println("After replace: " + sb);
```

```
Vellore Fort is in Vellore
erolleV ni si troF erolleV
Before delete: Vellore Fort is in Vellore
After delete: Vell Fort is in Vellore
After deleteCharAt: ell Fort is in Vellore
After replace: ell Fist is in Vellore
```

More Methods of class String Buffer

- String substring(int startIndex)
- String substring(int startIndex, int endIndex)
- char charAt(int where)
- void setCharAt(int where, char ch)
- void getChars(int sourceStart, int sourceEnd, char target[ ],int targetStart)

- Note: These methods are available in String class too.

String Builder

- Object that represents a sequence of char values.
- StringBuilder is a mutable Object.
- Not Synchronized and thus not thread safe.
- Faster in Performance.
- Introduced in JDK1.5 Version.
- Same as StringBuffer Class.

String Buffer Vs StringBuilder

Sl. No.	String Buffer	String Builder
1.	Synchronized	Non-Synchronized
2.	Thread Safe	Not Thread Safe
3.	Less Efficient	More Efficient
4.	Slower	Faster
5.	Java 1.0	Java 1.5

Summary

- What is a java Stringbuffer class?
- Constructors and Methods of StringBuffer
- What is a java Stringbuilder class?
- StringBuffer Vs StringBuilder
- Example program Snippets.

JAVA Wrapper Classes & Scope of Variables

- What is a Wrapper class?
- Need of Wrapper Class.
- Autoboxing and unboxing.
- Scope of Variables.
- Example program.

Wrapper Classes

- The wrapper classes in Java are used to convert primitive types (int, char, float, etc) into corresponding objects.
- The wrapper classes do not have no-arg constructors.
- The instances of all wrapper classes are immutable, i.e., their internal values cannot be changed once the objects are created.

Wrapper Classes

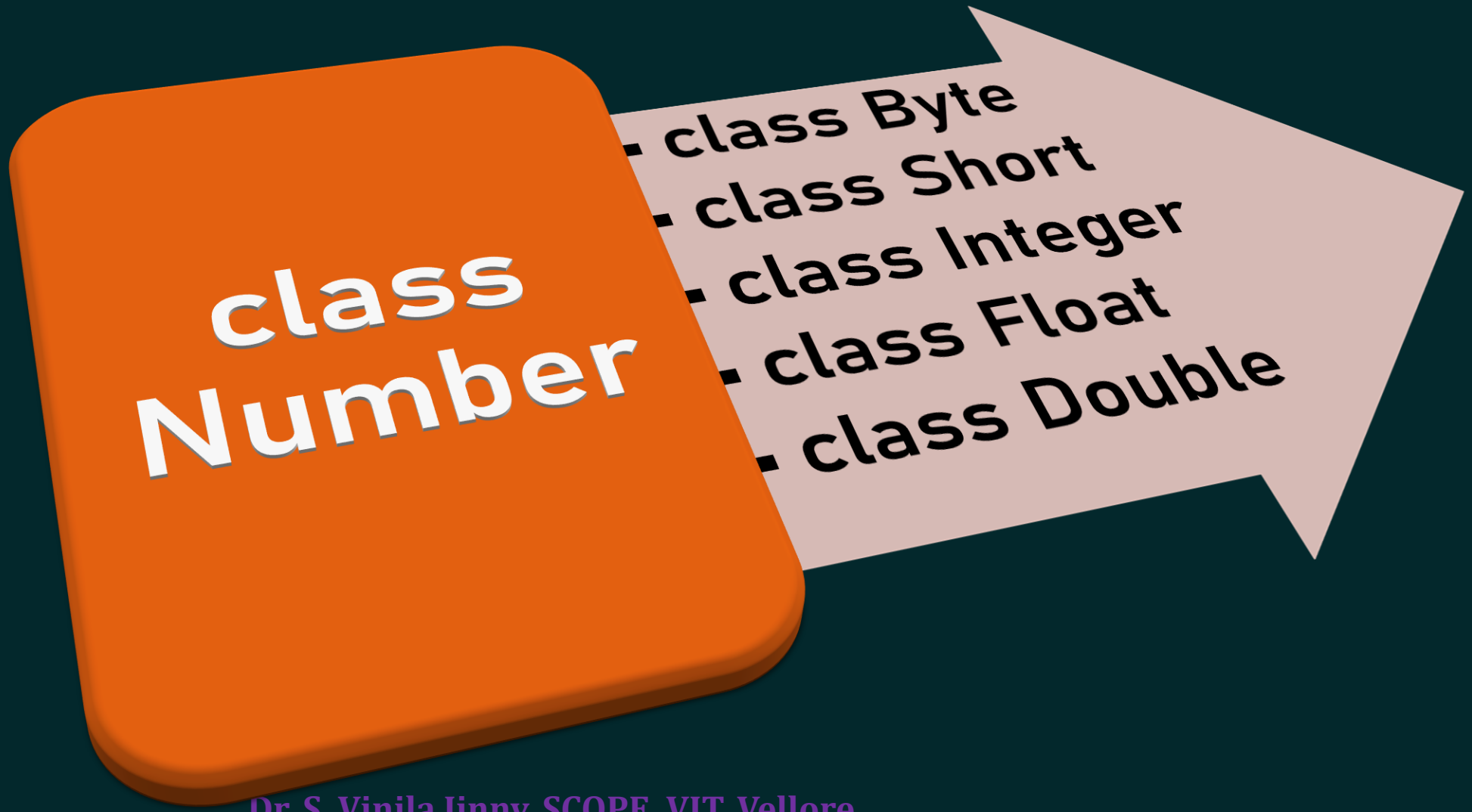
- The java.lang package contains a wrapper class that corresponds to each primitive type.
- Since J2SE 5.0, autoboxing and unboxing feature convert primitives into objects and objects into primitives automatically.
- The automatic conversion of primitive into an object is known as autoboxing and vice-versa unboxing

Wrapper Classes for converting simple types

Simple Type	Wrapper Class
byte	Byte
boolean	Boolean
char	Character
double	Double
float	Float
int	Integer
long	Long
short	Short

8 Wrappers

Wrapper Classes for converting simple types



Need of Wrapper Class

- Null Values
- Generics
- Method Overloading
- Data Structures
- Serialisation
- Synchronization
- Collection Framework

Wrapper Classes vs. Data Types

- Memory Overhead
- Immutability
- Auto-Boxing and Auto-Unboxing

Wrapper Classes

- The wrapper classes provide constructors, constants, and conversion methods for manipulating various data types.

Methods in Wrapper classes

- doubleValue()
- floatValue()
- intValue()
- longValue()
- shortValue()
- byteValue().

- These methods “convert” objects into primitive type values.

Methods in Wrapper classes

- doubleValue()
- floatValue(),
- intValue()
- longValue()
- shortValue()

- These methods “convert” primitive type into wrapper objects.

Constants in Wrapper classes

- Each numeric wrapper class has the constants MAX_VALUE and MIN_VALUE.
- MAX_VALUE represents the maximum value of the corresponding primitive data type.
- MIN_VALUE represents the minimum value of the corresponding primitive data type.

Frequently used Methods in Wrapper classes

- valueOf(): Creates a wrapper object from a given primitive value.
- parseInt(): Parses a string and returns an int (available in Integer, Double, etc.).
- toString(): Converts the wrapper object to a string representation.
- compareTo(): Compares two wrapper objects.
- equals(): Checks if two wrapper objects are equal.
- hashCode(): Returns a hash code for the object.

Autoboxing

- The automatic conversion of primitive data type into its corresponding wrapper class is known as autoboxing,
- Since Java 5, we do not need to use the valueOf() method of wrapper classes to convert the primitive into objects.

```
Integer intObject = new Integer (2);
```



```
Integer intObject = 2;
```

Unboxing

- The automatic conversion of wrapper type into its corresponding primitive type is known as unboxing.
- It is the reverse process of autoboxing.
- Since Java 5, we do not need to use the intValue() method of wrapper classes to convert the wrapper type into primitives.

Autoboxing

```
public class WrapperAutobox
{
public static void main(String args[])
{
int a=20;
Integer i=Integer.valueOf(a);
Integer j=a;
System.out.println(a+" "+i+" "+j);
}
}
```

Unboxing

```
public class WrapperUnbox{
public static void main(String args[])
{
Integer a=new Integer(3);
int i=a.intValue();
int j=a;
System.out.println(a+" "+i+" "+j);
}
}
```

Scope of Variables

- The scope of variables in Java is a location (or region) of the program where the variable is visible to a program and can be accessible.
- local variables
- instance variables
- class variables (static variables).

Local Variables

- A local variable is a variable declared inside a method body, block or constructor. It means variable is only accessible inside the method, block or constructor that declared it.
- Scope is inside the block only.

Instance Variables

- The variables that are declared inside the class but outside the scope of any method are called instance variables in Java.
- The instance variable is initialized at the time of the class loading or when an object of the class is created.
- An instance variable can be declared using different access modifiers available in Java like default, private, public, and protected.

Static Variables

- In Java, a static variable is a variable that is shared among all instances of a class.
- It is also known as a class variable.
- The scope of a static variable in Java is the entire class, and its lifetime is the duration of the program.

Example

```
public class ScopeTest
{
   int num = 20; // instance variable
   static int a = 20; // global or static variable
    void m1()
    {
            int a = 30; // local varakible
            System.out.println("a: " +a);
            System.out.println("a: " +StaticTest.a);
    }
   void m1()
   {
          int num = 30; // local variable
          System.out.println("Number: " +num);
          System.out.println("Number: " +this.num);
   }
public static void main(String[] args)
{
  ScopeTest st = new ScopeTest();
   st.m1();
 }
}
```

Summary

- What is a Wrapper class?
- Need of Wrapper Class.
- Autoboxing and unboxing.
- Scope of Variables.
- Example program.